# Sonder: Sequential Hypothesis Discrimination for Misconception Diagnosis in Grade-8 STEM


Project Team

| | |
|---|---|
| Syed Ashher Majid | 23I-0007 |
| Ahmed Laiq | 23I-0657 |
| Muhammad Harris Azhar | 23I-2535 |


Session 2023-2027


Supervised by

Dr. Ali Zeeshan Ijaz






**FAST School of Computing**

**National University of Computer and Emerging Sciences**
**Islamabad, Pakistan**


**October, 2026**

# Contents

**Students' Submission** i

**Anti-Plagiarism Statement** iii

**Authors' Declaration** v

**Abstract** vii

**Acknowledgments** ix

**Executive Summary** xi

**List of Abbreviations** xiii

**1 Introduction and Project Vision** 1

1.1 Project Overview . . . 1
1.2 Problem Statement . . . 1
1.3 Problem Context and Motivation . . . 2
1.4 Project Vision . . . 2
1.5 Project Objectives . . . 3
1.6 Project Scope . . . 3
1.7 Project Modules . . . 4
1.8 Stakeholders and User Classes . . . 5
1.9 Success Criteria . . . 5
1.10 Constraints and Assumptions . . . 5
1.11 Ethical, Legal and DEI Considerations . . . 6

**2 Research and Existing Solutions** 7

2.1 Research Method / Approach . . . 7
2.2 Existing Solutions / Products . . . 7
2.3 Comparison of Existing Solutions . . . 9
2.4 Limitations and Gaps . . . 9
2.5 Proposed Solution . . . 10
2.6 Complex Computing Problem . . . 10

2.7 Technical Challenges . . . 11
2.8 Research Findings and Design Decisions . . . 12

**3 Proposal Feedback and Iteration Plan 13**
3.1 Proposal Feedback Summary . . . 13
3.2 Proposal Action Register . . . 13
3.3 Iteration Strategy . . . 16
3.4 Iteration 1 Plan . . . 17
3.5 Iteration 2 Plan . . . 17
3.6 Work Breakdown and Responsibilities . . . 18
3.7 Progress Against Plan . . . 19
3.8 Delays, Deviations and Recovery Plan . . . 20

**4 Software Requirements 21**
4.1 User Interactions / Use Cases . . . 22
4.2 Use Case Diagram . . . 23
4.3 Detailed Use Cases . . . 24
4.4 Functional Requirements . . . 25
4.4.1 Module 1: Diagnostic engine . . . 25
4.4.2 Module 2: Content store . . . 26
4.4.3 Deferred product requirements . . . 26
4.5 Non-Functional Requirements . . . 26
4.5.1 Reliability . . . 26
4.5.2 Usability . . . 27
4.5.3 Performance . . . 27
4.5.4 Security . . . 27
4.6 Acceptance Criteria . . . 28
4.7 Requirements Traceability . . . 29

**5 System Design 31**
5.1 System Architecture . . . 31
5.2 Architectural Components . . . 32
5.3 Interfaces and Dependencies . . . 33
5.4 Architecture Decisions and Rationale . . . 34
5.5 Design Evolution . . . 35
5.6 Domain Model . . . 36
5.7 Data Design . . . 36
5.8 Process / Behavior Design . . . 41
5.8.1 Data flow: from reviewed questions to a frozen pack . . . 41
5.8.2 Data flow: one diagnostic step . . . 42
5.8.3 State diagram: session outcomes . . . 42

5.8.4 State diagram: content lifecycle . . . . . 43

**6 Iteration 1: Design and Implementation 45**
6.1 Iteration Objectives . . . . . 45
6.2 Selected Use Cases / Features . . . . . 45
6.3 Design . . . . . 45
6.4 Implementation . . . . . 46
6.5 Completed Features / Module . . . . . 46
6.6 Technical Implementation Details . . . . . 47
6.6.1 Setup and execution (to reproduce) . . . . . 47
6.6.2 Calculation . . . . . 47
6.6.3 Selection . . . . . 48
6.6.4 Stopping and refusal . . . . . 48
6.6.5 Pack loading . . . . . 49
6.7 Algorithms and Technical Decisions . . . . . 50
6.8 Tools, Frameworks and Technologies . . . . . 50
6.9 Demonstration Evidence . . . . . 51
6.9.1 Normal case: worked example D1 (unit U1, MATH-1379 student) 51
6.9.2 Edge case: worked example D2 (unit U4, inseparable pair) . . . . 51
6.9.3 The same examples in the browser . . . . . 52
6.9.4 Failure cases . . . . . 52
6.10 Iteration Outcome . . . . . 52

**7 Testing and Early Validation 53**
7.1 Testing Strategy . . . . . 53
7.2 Test Levels and Techniques . . . . . 54
7.3 Test Cases . . . . . 55
7.4 Test Results . . . . . 55
7.4.1 Performance . . . . . 56
7.5 Acceptance Testing / Validation . . . . . 57
7.6 Edge and Failure Cases . . . . . 57
7.7 Defects and Corrective Actions . . . . . 58
7.8 Current Validation Status . . . . . 58

**8 Project Management and Development Evidence 61**
8.1 Work Completed . . . . . 61
8.2 Work Remaining . . . . . 62
8.3 Individual Responsibilities . . . . . 62
8.4 Contribution of Each Team Member . . . . . 62
8.5 Repository and Version Control Evidence . . . . . 63
8.6 Milestones and Commit Evidence . . . . . 64

8.7 Tools and Tool-Assisted Development . . . 64
8.8 External Components and Dependencies . . . 66
8.9 Risks and Mitigation . . . 67
8.10 Security and Privacy Considerations . . . 67
8.11 Current Limitations . . . 68

**9 Remaining FYP Plan** **69**
9.1 Remaining Scope . . . 69
9.2 Remaining Modules and Features . . . 70
9.3 Remaining Iterations . . . 70
9.4 Implementation Plan . . . 71
9.5 Testing and Validation Plan . . . 71
9.6 Deployment Plan . . . 72
9.7 Tentative Timeline . . . 72
9.8 Expected Final Deliverables . . . 73

**10 Conclusion** **75**

**A Proposal Action Register** **77**

**B Detailed Requirements** **79**

**C Detailed Test Cases** **83**
C.1 Deterministic students through the run loop . . . 83
C.2 Injected faults (mutation check) . . . 83
C.3 Selection test classes (`tests/test_selection.py`) . . . 84
C.4 Commands to reproduce . . . 84

**D Repository and Contribution Evidence** **85**

**E Additional Design Diagrams** **87**

**F Meeting and Progress Evidence** **91**

# List of Figures


4.1 Use case diagram. Solid: built in Iteration 1 (engine and schema side). Dashed: later iterations. . . . . 23

5.1 Iteration 1 target architecture, status as of 12 October 2026. Built: the engine library (including the stopping and refusal rules, run loop and trace), CI and the live student answer flow through the engine service. In progress: saving runs to the database (runs are in memory now), the importer to Neon and the remaining interface screens. Planned for Mid-1 but not built: authentication, the teacher dashboard and class screens. Grey dashed boxes are later iterations. . . . . 31
5.2 Domain model of the diagnostic core. . . . . 36
5.3 ERD: Mathematics content pack (topics, misconceptions, items, options, Q-matrix links, sources, reviews). . . . . 37
5.4 ERD: diagnostic units (ordered hypothesis slots and item positions). . . . 38
5.5 ERD: engine model and frozen inputs (parameters, response model, probabilities, engine release). . . . . 39
5.6 ERD: diagnostic runs (session, hypotheses, questions, candidate ranking, steps, posteriors, diagnosis, escalation). . . . . 40
5.7 Flow 1: reviewed questions to a frozen pack. NONE, UNMODELLED and the units must exist before the freeze. . . . . 41
5.8 Flow 2: one answer-and-select step of a session. . . . . 42
5.9 Session state machine (database column `diagnostic_session.status`). All outcomes are terminal; rules are evaluated in order R1 to R5 after each update. . . . . 42
5.10 Content state machines enforced by the database (`state_guard`). Approval and rejection happen only by a review by a non-author. . . . . 43

E.1 ERD: users and access. . . . . 87
E.2 ERD: classes and enrolment. . . . . 88
E.3 ERD: audit. . . . . 89
E.4 Complete Iteration 1 schema (41 tables, revision 3). . . . . 90

# List of Tables


1.1 Stakeholder interview (one Grade-8 teacher). . . . . 2
1.2 Project modules and Iteration 1 state. . . . . 4
1.3 User classes. . . . . 5
1.4 Measurable success criteria and current status. . . . . 5

2.1 Comparison against the capabilities Sonder targets. . . . . 9
2.2 Why a Bayesian engine (assessment of candidate approaches against requirements). . . . . 10
2.3 Characteristics of the complex computing problem. . . . . 11
2.4 Findings and the decisions they produced. . . . . 12

3.2 Four iterations (from the proposal). . . . . 17
3.3 Technical ownership in Iteration 1. . . . . 18
3.4 Iteration 1 work packages: planned versus actual (evaluation commit `2c266d4` on `dev`, 12 October 2026). . . . . 19

4.1 Use cases by actor. "I1" marks use cases whose engine part is built in Iteration 1. . . . . 22
4.2 UC1 — Answer diagnostic questions (engine path). . . . . 24
4.3 UC5/UC6 — Review items and freeze a content pack. . . . . 24
4.4 Engine functional requirements. . . . . 25
4.5 Content functional requirements. . . . . 26
4.6 Product functional requirements outside Iteration 1. . . . . 26
4.7 Acceptance criteria and verification. . . . . 28
4.8 Requirement → design → implementation → test → owner. . . . . 29

5.1 Components and their Iteration 1 state. . . . . 32
5.2 Architecture decisions. . . . . 34
5.3 Design changes since the proposal. . . . . 35
5.4 Selected integrity rules enforced by the database. . . . . 41

6.1 Implemented modules (all under `sonder_engine/` unless stated). . . . . 46
6.2 Stopping and refusal rules (defaults: $\theta = 0.80$, $\theta_U = 0.50$, `min_support`=2, `min_support_none`=3, $\eta = 0.01$ bits, budget 8). . . . . 49
6.3 Technical decisions and open items (spec decision IDs). . . . . 50

6.4 Tools and technologies. . . . . 50
6.5 Worked example D1 on unit U1 (rounding). Top posterior entries after each answer. . . . . 51

7.1 Test levels. . . . . 54
7.2 Test inventory on the evaluation branch (186 Python tests; the interface has a further 123). . . . . 55
7.3 Deterministic synthetic students (specification §6.1, recomputed for 43 students). Students always apply the misconception when triggered and never slip. . . . . 56
7.4 Latency of one step (target: under 300 ms). . . . . 56
7.5 Edge and failure cases. . . . . 57
7.6 Defects found during the iteration and their handling. . . . . 58
7.7 Known faults of the engine and its data (full list with evidence: `sonder_engine/KNOWN_FAULTS.md`). . . . . 59

8.1 Contribution by member (pull requests merged into `dev`, counted by branch owner; #3 is a `dev` to `main` synchronisation). . . . . 62
8.2 Milestones (merge dates from the repository history). . . . . 64
8.3 GenAI register for materially assisted artefacts. . . . . 64
8.4 External components, risks and fallbacks. . . . . 66
8.5 Risk register. . . . . 67

9.1 Remaining modules. . . . . 70
9.2 Planned studies (from the proposal). . . . . 71
9.3 Relative timeline (months $M1$–$M8$ from the roadmap); calendar dates to be added. . . . . 72

A.1 Panel comments and evidence. . . . . 77

# Students' Submission

We certify that this report is submitted as the FYP-1 mid-report for the project titled *Sonder: Sequential Hypothesis Discrimination for Misconception Diagnosis in Grade-8 STEM*.

Student 1: Syed Ashher Majid (23I-0007) Signature: _____

Student 2: Ahmed Laiq (23I-0657) Signature: _____

Student 3: Muhammad Harris Azhar (23I-2535) Signature: _____

# Anti-Plagiarism Statement

The students certify that this report is their own work and that all sources used in preparing the report have been appropriately acknowledged and referenced.

# Authors' Declaration

We declare that the work documented in this report represents the work completed by the project team up to the FYP-1 mid-report stage. Contributions, tools, external components, and relevant sources are disclosed where applicable.

Generative AI was used in this project. Claude (Anthropic) drafted the Mathematics items and their misconception links, and Claude Code assisted with code, tests, diagrams and parts of this report. Each assisted artefact, the assistance level (L1 to L4), the team member who owns it and the way its output was checked are recorded in Section 8.7 and in the GenAI register of the evidence worksheet. The diagnostic calculation itself contains no generative model. Every member of the team can explain and modify the work attributed to them.

# Abstract

A wrong answer to a multiple-choice question is often consistent with more than one misconception, yet diagnostic tools usually map each wrong answer to a single fixed label and never signal their uncertainty. A student then receives remediation for a misconception they do not hold, and the teacher plans from an uncalibrated label. Sonder addresses this for Grade-8 Mathematics, Physics and Chemistry. It keeps a Bayesian posterior over a small set of competing misconceptions, chooses the next question that is expected to reduce that uncertainty most, and, when the evidence cannot support a diagnosis, ends the session with an explicit refusal instead of a forced answer. The diagnostic path is deterministic and contains no generative model, so every decision can be replayed and explained.

This mid-report covers Iteration 1, which is restricted to Mathematics. The team (i) built and reviewed a bank of 117 multiple-choice items with 28 misconceptions drawn from the Eedi catalogue, (ii) wrote an engine specification measured on that bank, (iii) implemented the engine as a pure Python library: prior, likelihood and posterior update, expected-information-gain selection, stopping and refusal rules, a run loop and an evidence trail, (iv) designed a 41-table PostgreSQL schema that stores frozen content, runs and audit records and is checked automatically in continuous integration, and (v) connected a live student Mathematics diagnostic in the user interface to the same engine through a local service. The code is verified by 186 Python tests, including exact-arithmetic and brute-force cross-checks and a harness of 43 deterministic synthetic students, and by 123 interface tests; one answer-and-select step takes a median of about 10 ms against a target of 300 ms.

The results are limited. The item bank is synthetic and no real student has answered it, so every probability and threshold is an assumed value, and the accuracy targets of the proposal (at most 5% confident misdiagnoses with at least 70% coverage) are not yet testable. Live runs are held in memory and are not yet saved to the database, there is no teacher result screen or login, and only Mathematics content exists. Fourteen known faults of the engine and its data are documented, each with the check that demonstrates it. FYP-2 will save runs to the database, add the teacher approval workflow and verification loop, extend the content to Physics and Chemistry, calibrate the parameters on real responses,

and evaluate against baselines.

# Acknowledgments

The project team thanks the project supervisor, Dr. Ali Zeeshan Ijaz, for guidance on scope and technical depth; the proposal panel, whose comments on coordination and technical challenge shaped this report; the teacher who gave the stakeholder interview; and the faculty of the FAST School of Computing. The misconception catalogue used to seed the Mathematics content is the one published by Eedi through the Kaggle competition "Mining Misconceptions in Mathematics".

# Executive Summary

**Problem and motivation.** In one teacher's words, "different reasons can lead to the same wrong answer". Existing tools commit to one misconception per wrong answer and never say when they are unsure, so students and teachers spend scarce time on the wrong remediation. Sonder is a teacher-supervised diagnostic platform for Grade-8 Mathematics, Physics and Chemistry that treats this ambiguity as the central computing problem: given a response consistent with several misconceptions, choose the next question that most reduces uncertainty under a fixed question budget, and refuse to diagnose when the evidence is insufficient.

**Scope.** The approved scope has four iterations. Iteration 1 (this report) delivers the diagnostic core for Mathematics only: a reviewed content pack, a deterministic engine, a database schema, a live student diagnostic and their tests. Teacher approval, intervention delivery, the verification loop, messaging, Physics and Chemistry content, an AI consultant and deployment follow in Iterations 2 to 4. The panel asked for more technical depth and for scope to be discussed with the supervisor; Chapter 3 records the response.

**Research findings.** Cognitive diagnostic computerised adaptive testing (CD-CAT) supplies the method; the closest published proposal (EDGE, 2025) describes such a system only in theory. A comparison of Eedi, MATHia, ALEKS, ASSISTments and DIAGNOSER (Chapter 2) shows that none selects follow-up questions to separate competing misconceptions or refuses explicitly. Measuring the real item bank changed the design: a session is scoped to a *diagnostic unit* (a declared hypothesis list and item list) rather than a topic, because many items cross topics.

**What has been built.** The Maths_v2 pack has 117 reviewed items (468 options) over 28 misconceptions; 114 items belong to seven diagnostic units. The engine, a pure Python library, performs the Bayesian update, expected-information-gain selection, the five stopping and refusal rules, a run loop and an evidence trail, and loads the pack through a validating loader. A local engine service and same-origin interface routes let a student take a real Mathematics diagnostic in the browser; the student sees only questions and a neutral completion message, and the result is kept for the teacher. A 41-table PostgreSQL schema with 110 triggers stores frozen content, runs and audit records; its rules are tested in continuous integration, which also loads all 117 items into it. The worked

example replays on the command line and in the browser: a student holding MATH-1379 is diagnosed after four answers with posterior 0.942.

**Validation status.** 186 Python tests and 123 interface tests pass on the integrated `dev` branch, including exact-fraction and brute-force checks of the selection, a harness of 43 deterministic synthetic students, and checks that the engine uses no AI, file, network, clock or random source. One answer-and-select step takes a median of about 10 ms (target 300 ms). Of 43 deterministic synthetic students, 26 are diagnosed correctly, 7 correctly show no misconception, none are misdiagnosed, eight are refused because two pairs of misconceptions are indistinguishable in the data, and two run out of useful questions.

**Risks and limits.** The bank is synthetic and all parameters are assumed, so accuracy and calibration are untested; 14 known faults are documented, among them inseparable misconception pairs, three skipped items and a high rate at which random clickers are diagnosed (about 40% at the default threshold). Live runs are held in memory and are not yet saved to the database; there is no login or teacher result screen, and the other interface screens are scripted. Contribution is uneven across the team, as the commit record shows.

**Remaining work.** Save runs and their evidence trail to the database, add the teacher result view and approval, resolve the open specification decisions, commit the noisy-student and random-clicker simulations as reproducible tests, calibrate on real responses, and deliver Iterations 2 to 4. Chapter 9 gives the plan.

# Contents

# List of Figures

# List of Tables

# List of Abbreviations

| | |
|---|---|
| FYP | Final Year Project |
| FR / NFR | Functional / Non-Functional Requirement |
| CD-CAT | Cognitive Diagnostic Computerized Adaptive Testing |
| EIG | Expected Information Gain (mutual information between the hypothesis and the next answer, in bits) |
| Q-matrix | Table linking each answer option to the misconception(s) it indicates |
| NONE | Engine hypothesis: the student holds no catalogued misconception |
| UNMODELLED | Engine hypothesis: the answers are not explained by any catalogued misconception |
| TV | Total variation distance between two answer distributions |
| P1, V5′, R1–R5 | Engine specification rules: response-model check, unit-membership rule, stopping and refusal rules |
| ERD | Entity-Relationship Diagram |
| CI | Continuous Integration |
| PR | Pull Request |
| GenAI / LLM | Generative AI / Large Language Model |
| RBAC | Role-Based Access Control |
| SDG | Sustainable Development Goal |

# Chapter 1

# Introduction and Project Vision

## 1.1 Project Overview

Sonder (formerly ConceptBridge) is a teacher-supervised platform that diagnoses specific misconceptions in Grade-8 Mathematics, Physics and Chemistry and helps students correct them. A student answers multiple-choice diagnostic questions. The system maintains a probability for each candidate misconception, picks the follow-up question that best tells the candidates apart, and either reports a diagnosis or refuses and escalates to a teacher. A teacher reviews each outcome before it reaches a student or parent. The planned product then delivers study material and re-checks the correction with a fresh question, immediately and after a delay.

The project belongs to Stream B of the FAST final-year-project scheme: the product has a bounded teacher workflow, a primary stakeholder (the teacher) and committed acceptance criteria, with the CD-CAT method as the research anchor of its core module. This report covers the FYP-1 mid stage, Iteration 1, in which only the Mathematics diagnostic core is built.

## 1.2 Problem Statement

Given a student response that is consistent with more than one misconception, select the next diagnostic item that maximises the expected reduction in posterior uncertainty under a fixed question budget, without ever presenting a false-confidence diagnosis.

## 1.3 Problem Context and Motivation

Fixed diagnostic quizzes map one wrong answer to exactly one pre-defined misconception and ask no adaptive follow-up. When two misconceptions produce the same wrong answer, the quiz still commits to one label. The student receives remediation for something they may not hold, and the teacher plans interventions from a label with no stated confidence.

A stakeholder interview with one teacher (Table 1.1) confirms the premise. The teacher checks working or asks the student to explain; records of misconceptions are mostly kept in memory; class-wide patterns are hard to see without a tool; and, asked whether the same wrong answers always mean the same misunderstanding, the answer was "No. Different reasons can lead to the same wrong answer." The teacher would double-check a tool that shows uncertainty at first and trust it more once it proved reliable. This is one teacher, not the two to three teachers and ten to twelve students the interviewee recommended, so it is a first data point and not a completed study.

Table 1.1: Stakeholder interview (one Grade-8 teacher).

| Question | Answer |
|---|---|
| How do you find out why a student got an answer wrong? | By checking their working or asking them to explain. |
| Do you keep a record of misconceptions? | Mostly it stays in the teacher's memory. |
| Do the same wrong answers always mean the same misunderstanding? | No. Different reasons can lead to the same wrong answer. |
| How much time is spent finding the reason behind mistakes? | Limited, as teachers have many students and tasks. |
| Would you trust a new tool that shows uncertainty? | Initially I would double-check it, but I would trust it more if it proved reliable. |

The work supports Sustainable Development Goal 4 (quality education) as motivation; the contribution claimed is computational, not an efficacy claim.

## 1.4 Project Vision

Sonder should let a teacher see, for any diagnosed student, which misconception the evidence supports, how strongly, which alternatives remain, and the question-by-question trail behind it, and should say plainly when the evidence is not enough. One diagnostic engine serves all three subjects; subject differences live in versioned content packs, not in code.

## 1.5 Project Objectives

O1. Maintain a Bayesian posterior over a unit's misconceptions, plus `NONE` and `UNMODELLED`, and update it after every answer.

O2. Select each next question by expected information gain, deterministically and with a stored explanation.

O3. End a session with a diagnosis, a "none detected" result or an explicit refusal (no separating question, budget exhausted, unmodelled), never a forced label.

O4. Store every question, candidate ranking, answer, posterior and decision as an evidence trail.

O5. Build versioned, reviewed content packs; Mid-1 target: a reviewed Mathematics pack.

O6. Meet the proposal's targets at evaluation time: at most 5% confident misdiagnoses with at least 70% coverage on held-out data; posterior update plus selection under 300 ms; Cohen's $\kappa \geq 0.7$ between the engine and an independent teacher.

## 1.6 Project Scope

**Included in the approved project.** The diagnostic engine; the content store (Q-matrix, misconception catalogue, question bank) for Mathematics, Physics and Chemistry; student, teacher, parent and admin views; teacher approval, correction and escalation; student interventions (study notes, video links); a verification loop with a delayed re-check; classes, join codes, announcements and direct messaging; a self-improving question-bank pipeline; an AI consultant, learning hub and concept tutor chat (bounded and read-only with respect to the diagnosis); containerised deployment with a 2 Mbps target; and a classroom pilot with consent.

**Built in Iteration 1 (Mid-1).** Mathematics only: the reviewed Maths_v2 pack, the engine library with stopping rules, run loop and evidence trail, a local engine service with a live student Mathematics diagnostic in the user interface, the database schema with automated checks, design artefacts, tests and the pack validator. The other interface screens remain the scripted proof of concept.

**Excluded.** Automatic curriculum generation; diagnosing every Grade-8 misconception; efficacy claims from a small pilot; real-time learning from live data of minors; clinical or psychological assessment; and any LLM or retrieval result determining the live diagnosis. For Mid-1 additionally: Physics and Chemistry content and every feature of Iterations 2 to 4.

## 1.7 Project Modules

Table 1.2: Project modules and Iteration 1 state.

| Module | Responsibility | State at mid-report |
|---|---|---|
| Diagnostic engine | Posterior update, EIG selection, stopping and refusal, run loop, evidence trail | Library built and tested; connected to the student screen through a local service |
| Content store | Misconception catalogue, items, options, Q-matrix, reviews, frozen packs | Maths_v2 pack and schema built; production import into the hosted database not yet done |
| Database | Versioned content, runs, audit, users and classes (41 tables) | Written and tested automatically in CI; live runs not yet saved to it |
| Product layer | Student, teacher, parent, admin screens; classes | Student Mathematics diagnostic live; other screens scripted |
| Teacher gate and interventions | Approve, correct, reject; study notes; verification loop | Iteration 2 |
| AI consultant, tutor, question-bank pipeline | Bounded assistance and content generation | Iterations 2 to 3 |
| Deployment and evaluation | Containers, low-bandwidth, security, pilot, baselines | Iterations 3 to 4 |

## 1.8 Stakeholders and User Classes

Table 1.3: User classes.

| Class | Characteristics and needs |
|---|---|
| Student (primary) | Grade-8; answers diagnostic questions, receives teacher-approved remediation; sees an explanation, not raw probabilities. |
| Teacher (primary) | Time-poor; needs the full evidence trail, binding authority to approve, correct or reject, and a queue of refusals to resolve. |
| Parent (secondary) | Wants a plain-language summary, only after teacher approval. |
| Admin / content reviewer | Maintains the catalogue and coverage, checks content provenance and views aggregate performance. |

## 1.9 Success Criteria

Table 1.4: Measurable success criteria and current status.

| Criterion | Target | Status |
|---|---|---|
| Confident misdiagnosis and coverage, evaluated together | $\leq 5\%$ confident misdiagnosis with $\geq 70\%$ coverage on held-out data | Not testable: no real-student data |
| Latency | Posterior update plus selection $<$ 300 ms | Met for the library and local service path: median about 10 ms per step |
| Teacher agreement | Cohen's $\kappa \geq 0.7$ engine vs. independent teacher | Not yet measured |
| Mid-1 (this report) | Worked examples reproduce; tests pass; invalid input rejected | Met for the engine and the student flow (Chapter 7) |

## 1.10 Constraints and Assumptions

- Time: eight months for the project; Iteration 1 was compressed (the repository was created on 9 October 2026 from existing documents).

- The item bank is synthetic: items were drafted with an LLM and reviewed by the team. No real student has answered it, so every probability (slip 0.10, guess 0.30, priors, thresholds) is assumed.
- Mid-1 content is Mathematics only; Physics and Chemistry are drafts outside the engine's inputs.
- The diagnostic path must be deterministic and auditable. Generated content cannot enter the live bank without human review.
- Deployment target: low-bandwidth (about 2 Mbps), low-end Android devices, containerised.
- A pilot with minors requires institutional approval, guardian consent and student assent, which have not yet been obtained.

## 1.11 Ethical, Legal and DEI Considerations

**Equity.** The 2 Mbps target and low-end device support keep the tool usable beyond well-connected schools; content is kept curriculum-neutral. **Transparency.** Refusals are disclosed to the teacher as outcomes with evidence, not hidden. **Privacy.** No student data has been collected; the proposal commits to pseudonymous identifiers, data minimisation, written guardian consent, student assent, withdrawal rights and formal ethics approval before any pilot. **Legal.** No copyrighted instrument text is redistributed; the Eedi catalogue is used to seed misconception names, and its licence terms are still to be recorded. **Responsible AI.** The generated items are reviewed; the engine never calls a model; GenAI use is disclosed in Section 8.7.

# Chapter 2

# Research and Existing Solutions

## 2.1 Research Method / Approach

Four activities informed the work. (1) *Stakeholder inquiry*: a structured interview with one Grade-8 teacher (Table 1.1). (2) *Product and literature review*: Eedi and Eedi Labs, Carnegie Learning MATHia, ALEKS, ASSISTments, DIAGNOSER, the CD-CAT literature [1; 2] and the 2025 EDGE proposal. (3) *Prior-project review*: six earlier FAST final-year projects (QTaleem F22-040-D, IntelliLearn F22-058-D, DANAAI F23-095-D, QWiz F22-002-D, CheckMate F25-282-D, LearnX F25-312-D), compared structurally. (4) *Measurement on real content*: the 117-item Maths_v2 bank was measured for separability, cross-topic links and trigger counts, and the engine was run against synthetic students; the results changed the design (Section 2.8). Each AI-assisted literature step was checked against the original source; citations still to be confirmed are highlighted in the bibliography.

The misconception sources are the Eedi catalogue [3; 4] for Mathematics and, for the later Physics and Chemistry packs, the force-concept and facet literature [5; 6] and the two-tier diagnostic tradition [7; 8]. Expected information gain follows the information-theoretic definition of Shannon [9].

## 2.2 Existing Solutions / Products

**Eedi / Eedi Labs.**
Multiple-choice diagnostic questions whose wrong options are mapped to misconceptions by experts. The catalogue and the NeurIPS 2020 Education Challenge data [4] are public; Mathematics only. Its 2026 AI-tutor expansion changes how remediation is delivered, not how the next question is selected.

**Carnegie Learning MATHia, ALEKS.**
Knowledge tracing and probabilistic adaptive assessment of what a student knows. They model knowledge state and pacing; they do not resolve competition between misconceptions.

**ASSISTments.**
Curriculum-aligned assignments with feedback; a reporting and assignment tool, not an evidence-traceable diagnostic loop.

**DIAGNOSER.**
Facet-coded distractors from physics education research; a fixed question set with no adaptive selection, and not a live commercial product.

**CD-CAT literature.**
Adaptive item selection for cognitive diagnosis models [1], building on rule-space ideas [2]. The method exists; the literature reports no deployed teaching loop that combines it with refusal and teacher oversight.

**EDGE (2025).**
A theoretical four-stage pipeline that proposes the combination Sonder implements, without an implementation. The proposal cites it as context only; a full citation is not held in the project repository.

## 2.3 Comparison of Existing Solutions

Table 2.1: Comparison against the capabilities Sonder targets.

| System | Misconception-level diagnosis | Adaptive follow-up to separate misconceptions | Explicit refusal | Teacher approval gate | Auditable decision trail |
|---|---|---|---|---|---|
| Eedi / Eedi Labs | Yes (fixed map) | No | No | Not assessed | Not assessed |
| MATHia, ALEKS | No (knowledge state) | Mastery-driven | No | Not assessed | Not assessed |
| ASSISTments | No | No | No | Teacher-led | Reports only |
| DIAGNOSER | Yes (facets) | No | No | Not assessed | Not assessed |
| CD-CAT papers | Model-dependent | Yes | Not studied | n/a | Not studied |
| EDGE (proposal) | Proposed | Proposed | Proposed | Proposed | Proposed |
| **Sonder** | Yes | Yes (EIG) | Yes (five outcomes) | Planned (Iteration 2) | Yes (trace) |

The ratings summarise the product documentation and papers the team reviewed; they are a qualitative reading, not a feature audit of the commercial products. The six prior FAST projects were reviewed in the same way: each is an assessment, quiz or tutoring system that commits to a fixed outcome per response; none implements sequential discrimination between competing misconceptions.

## 2.4 Limitations and Gaps

Existing tools map a wrong answer to one fixed misconception, or model knowledge instead of misconceptions, and none adds adaptive disambiguation between competing misconceptions with an explicit refusal. The gap is methodological (fixed-sequence versus sequential discrimination), not a claim about local classrooms. It is supported by the teacher interview, by product documentation and by the literature; its generalisation beyond one teacher is not yet shown.

## 2.5 Proposed Solution

Sonder consists of three layers. A *diagnostic engine* holds a Bayesian posterior, selects the next question by expected information gain and applies stopping and refusal rules. A *content store* holds the Q-matrix, misconception catalogue and question bank in versioned, reviewed, frozen packs. A *product layer* gives each role its own view, with teacher approval before any result is released and an explanation service that is read-only with respect to the engine. The engine is chosen over rule-based, one-shot LLM and retrieve-and-rerank alternatives because it is deterministic, fully auditable, needs no training data, handles slip and guess explicitly, supports adaptive questioning and can refuse (Table 2.2).

Table 2.2: Why a Bayesian engine (assessment of candidate approaches against requirements).

| Requirement | Rule-based | LLM classification | Bayesian |
|---|---|---|---|
| Handles uncertainty | Partly | Partly | Yes |
| Combines multiple answers | No | Weakly | Yes |
| Explicit confidence | No | No (uncalibrated) | Yes |
| Deterministic and auditable | Yes | No | Yes |
| Handles slip and guess | No | No | Yes |
| Supports adaptive questioning | No | No | Yes |
| Can explicitly refuse | Limited | No | Yes |
| No neural training required | Yes | No | Yes |

## 2.6 Complex Computing Problem

**Formal problem.** Given a response consistent with several misconceptions, select the next diagnostic item that maximises expected reduction in posterior uncertainty under a fixed question budget, without presenting a false-confidence diagnosis.

Table 2.3: Characteristics of the complex computing problem.

| Characteristic | Application to Sonder |
|---|---|
| Familiarity of issues | The closest published work (EDGE) proposes this only in theory; no implementation exists to adapt. |
| Beyond standard practice | Adaptive entropy selection has no deployed teaching-loop precedent. |
| Diverse stakeholders | Students need a diagnosis, teachers the full evidence trail, parents an answer; each needs a different view of one result. |
| Significant consequences | False confidence wastes scarce instructional time on the wrong remediation. |
| Conflicting requirements | Diagnostic confidence versus limited classroom time and question budget. |
| Depth of analysis | A diagnosis is never one number: confidence, questions spent and margin over the runner-up are weighed together. |
| Depth of knowledge | Applied Bayesian cognitive diagnosis, information theory and psychometrics. |

## 2.7 Technical Challenges

1. **Ambiguity is built into the data.** 286 of the 351 wrong options carry at least one misconception link, and some options carry two or three, so the engine must reason about shared evidence.

2. **Some misconceptions cannot be told apart.** Two pairs (MATH-1514/1773 and MATH-1074/1651) predict the same answers on every item of their units; the engine must detect this and refuse honestly.

3. **Cross-topic items.** 26 items link misconceptions from outside their topic, so a topic-scoped session cannot use them.

4. **Model validity.** With three linked options and guess 0.30, a predicted option becomes less likely than the unpredicted correct option, which breaks the model (rule P1).

5. **Refusing without over-refusing.** Thresholds trade confident misdiagnoses against coverage; random clickers must not be diagnosed too often.

6. **Determinism and auditability** with floating-point arithmetic, tie-breaking and a stored explanation for every choice.

7. **No real data.** Calibration and accuracy claims must wait for a pilot.

## 2.8 Research Findings and Design Decisions

Table 2.4: Findings and the decisions they produced.

| Finding | Decision |
|---|---|
| Cross-topic items are the point of the bank, not exceptions | Session scope is a *diagnostic unit* (hypothesis list plus item list), not a topic; 114 of 117 items are placed in seven units (spec v2 §2.1). |
| Twin items share a stem | After one item of a twin group is asked, its twins are ineligible, so evidence stays independent. |
| A three-option predicted set breaks the likelihood model | Rule P1: guess $< (4-s)/4$ for predicted-set size $s$; items that violate it at the default parameters are skipped at load (decision D-2). |
| Inseparable pairs exist | Detected at load (warning W1) and reported as a refusal naming both; merging the pairs is recommended (D-1). |
| About 90 links are only "stretch" | Optional link grades S/T with weights 1.0 and 0.5. |
| Random answering is diagnosed often | Measured and documented; raising `min_support` to 3 halves the rate (D-3, open). |
| Database integrity matters for audit | Freezing, review and append-only rules are enforced by the database, not only by application code. |

# Chapter 3

# Proposal Feedback and Iteration Plan

## 3.1 Proposal Feedback Summary

The proposal panel recorded the following comments: “Coordination is weak between students; everyone needs to put in effort, not just one member. There is currently a weak technical challenge; more features with technical challenges should be included and the scope expanded. Overall, the presentation was good, the work is good, and the project idea is clearly presented. The system needs further development by incorporating additional features to enhance its usefulness for both students and teachers. Discuss scope with your supervisor.”

## 3.2 Proposal Action Register

| No. | Feedback / Required Change | Action Taken | Status | Evidence |
|---|---|---|---|---|
| 1 | Coordination is weak; everyone needs to put in effort, not just one member. | Accepted. Work is split by named owner with lead and reviewer pairs, and every member has merged work through pull requests. Honest status: contribution on `dev` is uneven (Section 8.4). Each pull request is reviewed by the reviewer the work division names (Harris reviews Ahmed's, Ahmed reviews Ashher's, Ashher reviews Harris's); a weekly contribution check and one non-author approval for merges into `dev` are to be agreed by 2026-10-18 (owner Ahmed). | In progress | Work-division document; CONTRIBUTING.md; merged pull requests #1 to #38 (Ahmed 21, Ashher 8, Harris 5, plus #3) |

| No. | Feedback / Required Change | Action Taken | Status | Evidence |
|---|---|---|---|---|
| 2 | Weak technical challenge; include more features with technical challenges. | Adapted. The core challenge is kept and deepened: a Bayesian posterior over up to eight competing misconceptions per unit, EIG selection with deterministic tie rules, five explicit stopping and refusal outcomes, link-grade weighting, and frozen versioned packs with an audit trail. The other technically hard proposal features (verification loop with delayed re-check, self-improving question bank, retrieval-based teacher consultant) are scheduled in Iterations 2 to 3. | In progress | Engine specification v2; engine library and tests (Chapters 5 to 7) |
| 3 | Add features that make the system more useful for students and teachers. | Accepted in part. The proposal already commits to teacher approval and escalation, interventions, verification, announcements and messaging, parent summaries and an AI consultant. Mid-1 delivers the Maths diagnostic slice first; which features move earlier is to be decided with the supervisor. | Open | Proposal Iteration Plan (Section 3.3); roadmap |

| No. | Feedback / Required Change | Action Taken | Status | Evidence |
|---|---|---|---|---|
| 4 | Discuss scope with your supervisor. | Accepted. A scope meeting with Dr. Ali Zeeshan Ijaz was held on 9 September 2026 to improve the technical challenge; the agreed scope is to be written into the roadmap (docs/ROADMAP.md). | Open | Roadmap; Appendix F |

## 3.3 Iteration Strategy

Development proceeds in four iterations, each cutting across the interface, content and engine tracks, and each ending at a FAST assessment gate. The first fixes the diagnostic core on Mathematics so that the central contribution is real before product features are layered on.

Table 3.2: Four iterations (from the proposal).

| Iteration | Content | Gate |
|---|---|---|
| 1 | Core diagnostic engine (Bayesian update, entropy selection); initial dashboards and classes; three-subject dataset curation and validation (Mathematics first) | FYP-1 Mid-1 (this report) |
| 2 | Teacher approval, correction and escalation; student interventions; verification loop and delayed re-check; self-improving question bank; second validation pass | FYP-1 Final-1 |
| 3 | AI consultant; learning hub and tutor chat; parent summaries; admin monitoring; engine evaluation; classes extended with announcements and messaging | FYP-2 Mid-2 |
| 4 | Containerised deployment, 2 Mbps optimisation; security hardening and RBAC audit; latency and stress tests (300 ms); classroom pilot; dataset freeze, audit and final report | FYP-2 Final-2 |

## 3.4 Iteration 1 Plan

**Objective.** An answer from the reviewed Mathematics subset changes the posterior by the implemented rule; the next item is chosen by computed information gain; tie, no-separator and budget states are handled explicitly; and a reviewer can see the result and evidence trail in a flow connected to the interface. **Deliverables and evidence** are the seven work packages in Table 3.4.

## 3.5 Iteration 2 Plan

Teacher approval, correction and escalation handling; student intervention delivery (study notes and video links); the verification loop with a delayed re-check scheduler; the self-improving question-bank pipeline (gap scan, generator, distractor engine); a second validation pass; first V1 simulation study; first deployment attempt. Evidence will be a working end-to-end core flow for FYP-1 Final-1. The exact content of Iteration 2 depends on

the scope agreement with the supervisor (action 4).

## 3.6 Work Breakdown and Responsibilities

Table 3.3: Technical ownership in Iteration 1.

| Member | Ownership |
|---|---|
| Muhammad Harris Azhar | Maths_v2 reviewed question pack; expected-information-gain question selection (`selection.py`); engine specification v2; ERDs and data-flow design |
| Syed Ashher Majid | Bayesian prior, likelihood and posterior (`bayes.py`, `model.py`); link grades and twin filtering; dataset validator and independent review; engine tests |
| Ahmed Laiq | Repository, CI and contribution rules; engine specification draft; database schema, tests and tools; architecture diagram; pack loader, stopping rules, run loop and evidence trail |

## 3.7 Progress Against Plan

Table 3.4: Iteration 1 work packages: planned versus actual (evaluation commit `2c266d4` on `dev`, 12 October 2026).

| Package | Owner | State | Evidence and variance |
|---|---|---|---|
| I1-01 Repository and ownership | All; Ahmed (CI) | Partial | Repository, branch and PR flow, five CI jobs (interface, dataset, engine, database schema, engine API); pull requests #1 to #38 (35 merged into `dev`). Planned date: none set; delivered 2026-10-10 (PRs #1 to #3). No task board is used; work is tracked by the task IDs in the work-division document and by pull requests. |
| I1-02 Requirements and design baseline | All | Partial | Specification v2 with amendments (PR #28), schema revision 3, ERDs and data flow (PRs #11, #26), architecture diagram (PRs #21, #36), state diagrams (Chapter 5), requirement IDs and traceability (Chapter 4). Ashher commits a repository copy of the state diagram on 2026-10-13. |
| I1-03 Maths dataset | Harris, Ashher | Partial | 117 approved items, 28 misconceptions (PR #5); validator and independent review (PR #6). Open: pack decisions M1 to M5, batch B09 left as a draft, no bank-wide independent human spot-check (Ashher reviewed T1); owners Harris and Ashher, 2026-10-25. |
| I1-04 Real Maths diagnostic calculation | Ashher, Harris, Ahmed | Done | Posterior update, EIG selection, five stopping rules, run loop, evidence trail and pack loader implemented and tested (PRs #7–#9, #20, #22–#24, #29, #31–#33); delivered 2026-10-11; 186 Python tests. Open: decisions D-1 and D-6; every parameter is assumed. |
| I1-05 Interface-connected live flow | Ahmed, all | Done (local checkpoint) | A student takes a real Mathematics diagnostic in the browser through same-origin routes and a local engine service (PR #35, 2026-10-12). Runs are held in memory and not yet saved to the database; there is no login or teacher result screen. Recovery: save runs and traces to the session tables (2026-10-18) and add a teacher result view (2026-10-25), owner Ahmed. |
| I1-07 Early validation | All | Done for the engine and the student flow | 186 Python tests, 123 interface tests, database schema checks 15 of 15 in CI (PR #34), end-to-end HTTP check (PR #35), synthetic-student harness. The browser walkthrough is checked by hand. Recovery: add tests for each new screen as it is built. |

## 3.8 Delays, Deviations and Recovery Plan

**Delay.** Three items are not complete: saving live runs to the database, a teacher result view, and a real login. The engine specification was re-measured on the real bank (it replaced an earlier draft written against an 80-item snapshot), the schema moved to its third revision, and the engine run loop and the live student flow were merged only on 11 and 12 October 2026. **Recovery.** (1) Import the pack into the hosted database from the existing probe script and save each run and its trace (owner Ahmed, 2026-10-18); (2) add a teacher result view (2026-10-25); (3) agree a weekly contribution check (2026-10-18). **Process deviation.** Integration work was concentrated in the last days: 26 of the 35 pull requests merged into `dev` were merged on 11 October 2026 and four more on 12 October. This is a coordination risk and is addressed by the weekly contribution check.

# Chapter 4

# Software Requirements

Requirements are numbered R1 to R8 (functional R1 to R6, non-functional R7 and R8) and refer to the proposal requirements FR-01 to FR-05 and its non-functional targets. Each carries a measurable acceptance criterion; statuses are those of the integrated `dev` branch on 11 October 2026.

## 4.1 User Interactions / Use Cases

Table 4.1: Use cases by actor. “I1” marks use cases whose engine part is built in Iteration 1.

| ID | Actor | Interaction | Iteration |
|---|---|---|---|
| UC1 | Student | Answers diagnostic questions on a unit; each answer updates the posterior and triggers the next question or a result | I1 (engine and student screen) |
| UC2 | Teacher | Inspects the evidence trail of a run: questions, ranking, reasons, answers, posteriors | I1 (trace; terminal view) |
| UC3 | Teacher | Resolves a refused (escalated) run using the tied candidate set | Iteration 2 |
| UC4 | Teacher | Approves, corrects or rejects an engine diagnosis before release | Iteration 2 |
| UC5 | Content reviewer | Reviews items (not the author), approves or rejects them | I1 (schema, data) |
| UC6 | Content reviewer / admin | Freezes a content pack and its response model | I1 (schema) |
| UC7 | Student, teacher | Creates or joins a class by code | Schema in I1; screens later |
| UC8 | Parent | Reads a plain-language summary after teacher approval | Iteration 3 |
| UC9 | Teacher | Sends announcements and direct messages | Iteration 3 |

## 4.2 Use Case Diagram

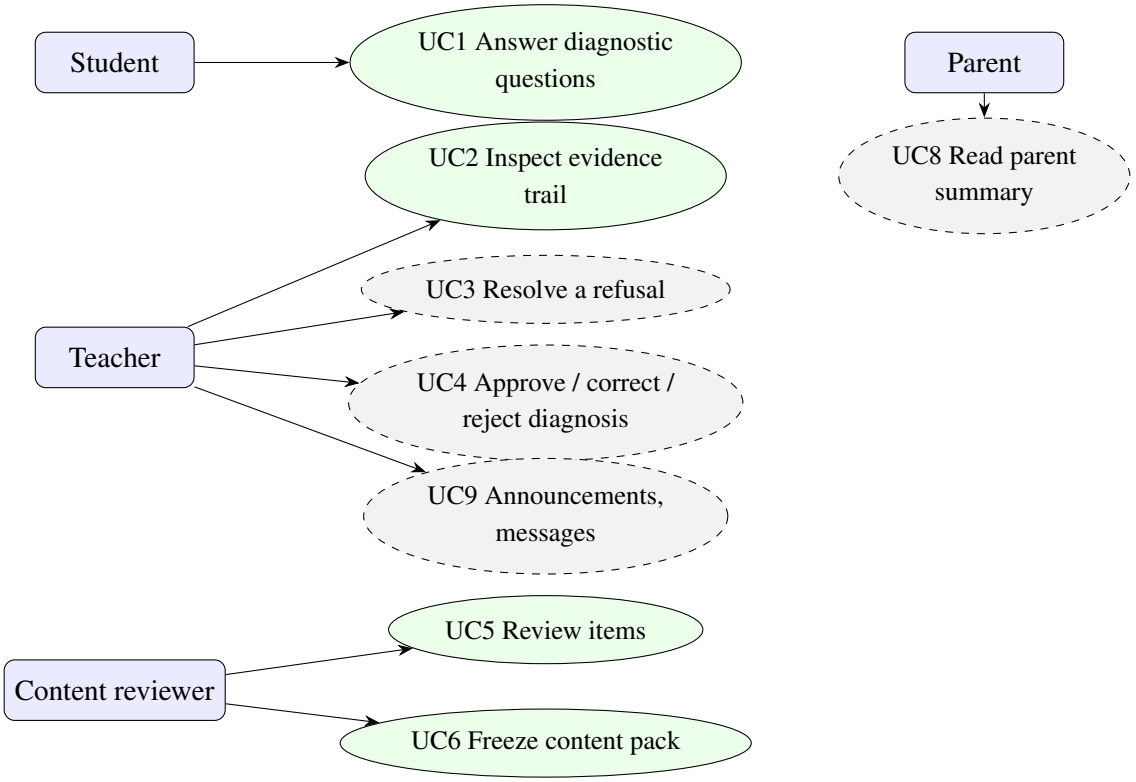



Figure 4.1: Use case diagram. Solid: built in Iteration 1 (engine and schema side). Dashed: later iterations.

## 4.3 Detailed Use Cases

Table 4.2: UC1 — Answer diagnostic questions (engine path).

| | |
|---|---|
| Actor | Student (through the interface); the engine is called by the service layer |
| Precondition | A frozen content pack and a pinned parameter set exist; the student is enrolled; a diagnostic unit is chosen |
| Main flow | 1. A session starts with the prior over the unit's hypotheses. 2. The engine chooses the eligible item with the highest expected information gain and stores the ranking and reason. 3. The student picks an option. 4. The engine applies the Bayes update and evaluates the stopping rules R1–R5. 5. If the outcome is `in_progress`, return to 2. 6. Otherwise the session ends with its outcome and trace. |
| Alternative flows | Invalid option or zero-probability evidence: the request is rejected and the session state is unchanged. Budget (eight questions), no remaining items, or best EIG below 0.01 bits: `escalated_exhausted`. No remaining item separates the leading hypotheses: `escalated_no_separator`. |
| Postcondition | The trace (questions, candidates, answers, posteriors, outcome) is complete and replayable from the option sequence alone |

Table 4.3: UC5/UC6 — Review items and freeze a content pack.

| | |
|---|---|
| Actor | Content reviewer (active teacher or admin, never the item's author) |
| Main flow | 1. Items are drafted, then submitted for review. 2. A reviewer records a review for the next round, approving or rejecting; approved items lock. 3. When every item is approved and every unit satisfies the pack rules, the pack is frozen with a SHA-256 hash. 4. The response model (probabilities from slip and guess) is frozen against it. |
| Alternative flows | A freeze with a draft or in-review item, a missing source, a missing NONE/UNMODELLED hypothesis or a rule violation is rejected by the database. |
| Postcondition | A frozen pack and response model are immutable and are the only inputs a run may use |

## 4.4 Functional Requirements

### 4.4.1 Module 1: Diagnostic engine

Table 4.4: Engine functional requirements.

| ID | Requirement | Source | Status |
|---|---|---|---|
| R1 | The engine selects the next item by maximising expected information gain over the live posterior; ties go to the earliest item in pack order; it returns the full ranking, entropy before and a generated reason | FR-01 | Met (library) |
| R2 | Insufficient evidence is a first-class outcome: a run ends as `diagnosed`, `none_detected`, `escalated_unmodelled`, `escalated_no_separator` or `escalated_exhausted`, with an evidence trail and, for refusals, the tied candidate set | FR-02 | Met (engine); results not yet saved to the database or shown on a teacher screen |
| R3 | One engine serves every subject with no subject-specific branching; subject differences live in packs | FR-03 | Partial: no branching in code; only the Maths pack exists |
| R9 | A student takes a Mathematics diagnostic in the user interface that is computed by the real engine, can resume after a refresh, and never sees the result (replies carry no diagnosis, posterior, rule or correct answer; a lost reply is retried without counting the answer twice) | Task T07.02 | Met (local checkpoint) |

### 4.4.2 Module 2: Content store

Table 4.5: Content functional requirements.

| ID | Requirement | Source | Status |
|---|---|---|---|
| R6 | Content packs are versioned, reviewed and frozen. An item is approved only by a recorded review from someone other than its author; a frozen pack is hashed and immutable; the engine reads only frozen packs | Proposal; trust requirement | Partial: schema rules, pack loader and a CI import check built; production import into the hosted database not done |

### 4.4.3 Deferred product requirements

Table 4.6: Product functional requirements outside Iteration 1.

| ID | Requirement | Source | Status |
|---|---|---|---|
| R4 | No diagnosis reaches a student or parent before a teacher approves, corrects or rejects it | FR-04 | Deferred (approval flow, Iteration 2); the student view already hides every result |
| R5 | Class announcements, direct messaging and parent–teacher communication respect role and class access rules | FR-05 | Deferred (Iteration 3) |

## 4.5 Non-Functional Requirements

### 4.5.1 Reliability

R8: the calculation is deterministic and rejects invalid input. The same pack, parameters and answers give identical posteriors, selections, reasons and decisions; invalid options raise `invalid_option`, impossible evidence raises `zero_evidence`, and a rejected request leaves the session unchanged. A negative information gain beyond $-10^{-12}$ is treated as a defect.

### 4.5.2 Usability

Teachers must read a decision without the probabilities being a black box: every selection carries a generated sentence stating the pair being separated and what each predicts. Student and parent views show an explanation, not raw posteriors. A usability study and the low-bandwidth target (a usable flow around 2 Mbps with session recovery) are deferred to Iterations 3 and 4.

### 4.5.3 Performance

R7: one posterior update plus question selection completes in under 300 ms. Measured on the integrated engine (Section 7.4.1): median about 10 ms and 95th percentile about 23 ms for one full step; the measurement excludes the database and the network.

### 4.5.4 Security

The engine has no I/O, network, clock or random source (checked by tests). Pseudonymous identifiers, role-scoped access and an append-only audit are required of the product; the schema defines roles, hashed credentials, append-only run records and an audit table. The local engine service requires a shared token, owns each run through a signed HttpOnly cookie, checks the request origin and listens on the loopback address only; there is no real login and no deployed service yet, and an RBAC audit is planned for Iteration 4. No secrets are committed to the repository.

## 4.6 Acceptance Criteria

Table 4.7: Acceptance criteria and verification.

| ID | Acceptance criterion | Verification |
|---|---|---|
| R1 | EIG agrees with exact arithmetic to $10^{-11}$; choice and ranking agree with a brute-force reference on 300 random posteriors; the specification worked examples reproduce to four decimals (D1: T1-005 at 0.7887, then T1-006, T1-007, T1-009; D2: T4-001 0.7517, T5-011 0.3668); an empty candidate list returns `None`; asked items and their twins are excluded | `tests/test_selection.py` (40 tests) |
| R2 | Each of the five stopping outcomes is produced by a constructed case; refusals name the tied set; a student with no misconception ends `none_detected`; MATH-1514/1773 end `escalated_no_separator` naming both | `tests/test_engine_stopping.py` (22), `test_engine_students.py` |
| R3 | No subject conditionals in the engine package; a pack of another subject loads unchanged | Source inspection; `tests/test_engine_no_ai.py`; second pack pending |
| R9 | Browser answers produce the engine's next item; a refresh resumes the same question; replies carry no diagnosis, posterior, rule or correct answer; a repeated request is not counted twice | `tests/test_engine_api.py` (16), 123 interface tests, `scripts/check_engine_ui.py` (D1, D2) |
| R6 | Invalid items and units are rejected by rules V1–V11; Maths_v2 loads into seven units with every linked misconception inside its unit (V5′); schema rejects invalid edits | `tests/test_engine_pack.py` (40); `behaviour_test.sql` (75 rejected writes and 15 of 15 checks, run in CI) |
| R7 | Median and 95th percentile of one step $<$ 300 ms | Timing script, Section 7.4.1 |
| R8 | Repeat runs identical; invalid input rejected with a named error | `tests/test_engine_session.py` (12), `test_bayes.py` (19), `test_engine_trace.py` (5) |
| R4, R5 | Teacher gate and messaging acceptance tests | Planned (Iterations 2 and 3) |

## 4.7 Requirements Traceability

Table 4.8: Requirement → design → implementation → test → owner.

| ID | Design | Implementation | Test | Owner |
|---|---|---|---|---|
| R1 | Spec §3.4; Section 6.6.3 | `selection.py` | `test_selection.py` | Harris |
| R2 | Spec §3.5; Figure 5.9; tables `diagnosis`, `escalation` | `stopping.py`, `session.py`, `trace.py` | `test_engine_stopping.py`, `test_engine_students.py` | Ahmed |
| R3 | Spec §2.1 | whole `sonder_engine` package (no subject branches) | `test_engine_no_ai.py` | Ashher |
| R4 | Teacher review tables, planned | approval not implemented; the student view hides every result (`sonder_api/student_view.py`) | `test_engine_api.py` | Iteration 2; Ahmed (student view) |
| R6 | Schema plan §4; Section 5.8 | `pack.py`, `sonder_pack/`, schema SQL | `test_engine_pack.py`, `behaviour_test.sql` | Ahmed, Harris (data) |
| R7 | Proposal NFR | `session.py` | timing script | Harris |
| R8 | Spec §2.3 | `bayes.py`, `session.py`, `selection.py` | `test_bayes.py`, `test_selection.py`, `test_engine_session.py` | Harris, Ashher |
| R9 | `sonder_api/README.md`; architecture diagram | `sonder_api/`; interface routes and screens | `test_engine_api.py`; interface tests; `check_engine_ui.py` | Ahmed |

# Chapter 5

# System Design

## 5.1 System Architecture

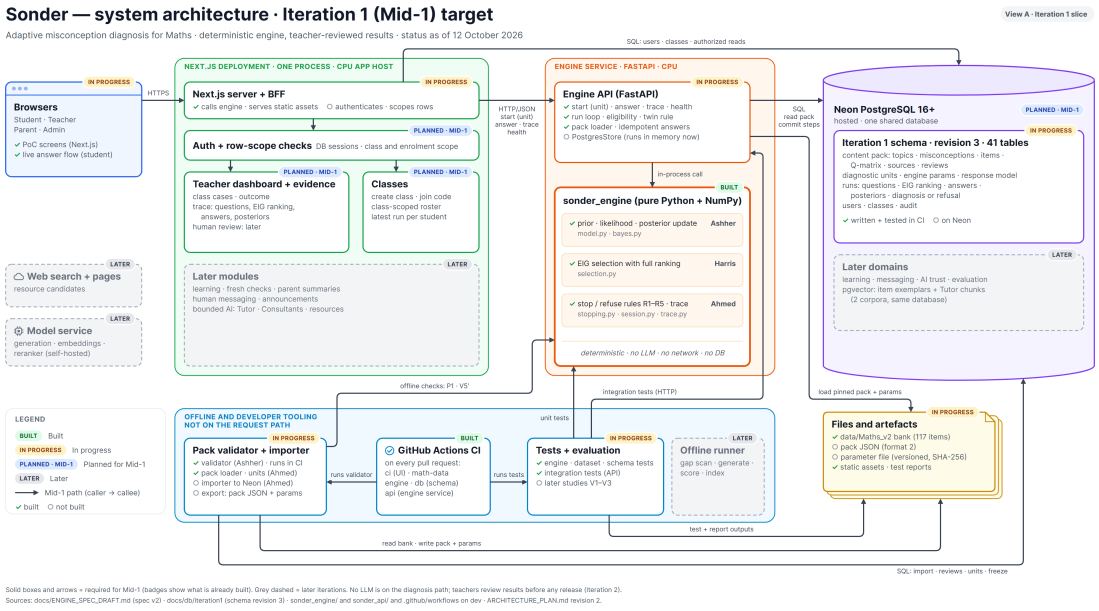



Figure 5.1: Iteration 1 target architecture, status as of 12 October 2026. Built: the engine library (including the stopping and refusal rules, run loop and trace), CI and the live student answer flow through the engine service. In progress: saving runs to the database (runs are in memory now), the importer to Neon and the remaining interface screens. Planned for Mid-1 but not built: authentication, the teacher dashboard and class screens. Grey dashed boxes are later iterations.

The system has four parts joined by one-directional interfaces. The browser talks to a Next.js server, which authenticates and scopes access, and calls an engine service over HTTP/JSON. The engine service runs a pure Python library, `sonder_engine`, in process;

the library performs the calculation and has no I/O, no network, no model call and no database access. A single PostgreSQL 16 database holds the frozen content, the run records and the audit trail. Offline tools (the pack validator, CI and tests) are not on the request path. The AI consultant and other model-backed services of later iterations are read-only with respect to the engine: they can explain a result but cannot write a posterior.

## 5.2 Architectural Components

Table 5.1: Components and their Iteration 1 state.

| Component | Responsibility | State |
|---|---|---|
| `sonder_engine` (library) | Prior, likelihood, posterior (`bayes.py`, `model.py`); eligibility and EIG selection (`selection.py`); stopping and refusal (`stopping.py`); run loop (`session.py`); evidence trail (`trace.py`); unit loader and rules (`pack.py`) | Built, 186 Python tests in total |
| `sonder_pack` | Reads Maths_v2 files, builds units U1–U7, prints warnings, replays a session on the command line | Built |
| Validator (`validation/`) | Checks the CSV bank and its source batches | Built, in CI |
| Database schema | 41 tables, 110 triggers, 1 view in seven domains: users, classes, content pack, diagnostic units, engine model, runs, audit | Written; 15 of 15 checks run in CI on PostgreSQL 16; live runs not yet saved to it |
| Engine service `sonder_api` (FastAPI) | Start, answer and trace endpoints; pack loader; signed run cookie, shared token and origin check; runs held in memory | Built (local); storage in the database to do |
| Next.js server and interface | Same-origin routes to the engine service; student diagnostic screens; role-based screens, authentication, class flows | Student Mathematics diagnostic live; other screens scripted |
| CI (GitHub Actions) | Five jobs: interface build and tests; dataset validator; engine tests; database schema checks in a throwaway PostgreSQL container; engine API tests | Built |

## 5.3 Interfaces and Dependencies

**Content to engine.** The only way content reaches the engine is a frozen pack: units with ordered hypotheses and items, options with graded misconception links, and a versioned parameter set. **Engine to storage.** The service writes each presented question with its full candidate ranking, each answer with its posterior, and the outcome in one transaction; database triggers check the writes but never calculate. **Engine public API** (`sonder_engine`): `prior`, `likelihood_matrix`, `update_posterior`, `eligible_items`, `expected_information_gain`, `select_next`, `check_stop`, `start`, `next_question`, `answer`, `replay`, `trace_of`. A session is an immutable value; every function returns a new state, and a rejected request leaves the state unchanged. **Live student path (built).** The browser calls same-origin Next.js routes, which call the local engine service; the service runs the unchanged engine, owns each run through a signed HttpOnly cookie, checks the request origin and a shared token, and returns only the next question or a neutral completion message. A retried answer is not counted twice. **Dependencies.** NumPy $\geq$ 2.0 (the only third-party package of the engine), Python $\geq$ 3.11, FastAPI, uvicorn and httpx for the optional service, PostgreSQL 16+ (hosted on Neon), Docker for the schema checks, Node for the interface.

## 5.4 Architecture Decisions and Rationale

Table 5.2: Architecture decisions.

| Decision | Alternatives | Chosen | Trade-off |
|---|---|---|---|
| Diagnosis method | Fixed quiz; rule-based; one-shot LLM; retrieve-and-rerank | Bayesian posterior with EIG selection | Deterministic, auditable, needs no training; depends on assumed slip and guess until calibrated |
| Scope of one session | Per topic; whole subject; per diagnostic unit | Diagnostic unit | Crossing items stay usable; units are defined by hand and limited to eight hypotheses |
| Engine structure | Logic in SQL; logic in the web app; pure library | Pure Python library, no I/O | Easy to test and reproduce; needs an integration layer |
| Where integrity rules live | Application code; database constraints and triggers | Database enforces freeze, review, append-only and trace rules | Strong guarantees; a heavy schema that is hard to change (slimming is an open team decision) |
| Handling ambiguity | Always return the top hypothesis; threshold only; explicit refusal | Explicit refusal outcomes with a separability rule | Lower coverage in exchange for no false-confidence diagnosis |
| LLM role | In the diagnostic path; outside it | Outside: drafting content only, with human review | Slower content growth; the diagnosis stays reproducible |

## 5.5 Design Evolution

Table 5.3: Design changes since the proposal.

| Area | Change | Why |
|---|---|---|
| Session scope | Topic → diagnostic unit (hypothesis list plus item list) | 26 of 117 items cross topics; a topic-scoped session cannot use them |
| Hypothesis limit | $K_{\max}$ 6 → 8 | One unit needs seven misconceptions |
| Twin items | Added twin groups: asking one makes its twins ineligible | Repeating a stem gives no independent evidence |
| Response model | Rule P1 generalised: guess $< (4 - s)/4$ | A three-option predicted set made a misconception holder prefer the correct answer |
| Link strength | Added link grades S/T with weights 1.0 and 0.5 | About 90 links are plausible but weak |
| Schema | Planning model → 41 tables (three revisions): diagnostic units, candidate ranking, engine parameters, response model | Spec v2 findings; stored ranking for the audit trail |
| Engine shape | Added stopping rules R1–R5, run loop, evidence trail and loader | Needed for a complete session and the refusal requirement |
| Product access to the engine | Added a local engine service behind same-origin interface routes; the student sees only questions and a neutral completion message | One engine implementation, no secrets in the browser, and every result stays behind the teacher gate (R4) |

## 5.6 Domain Model

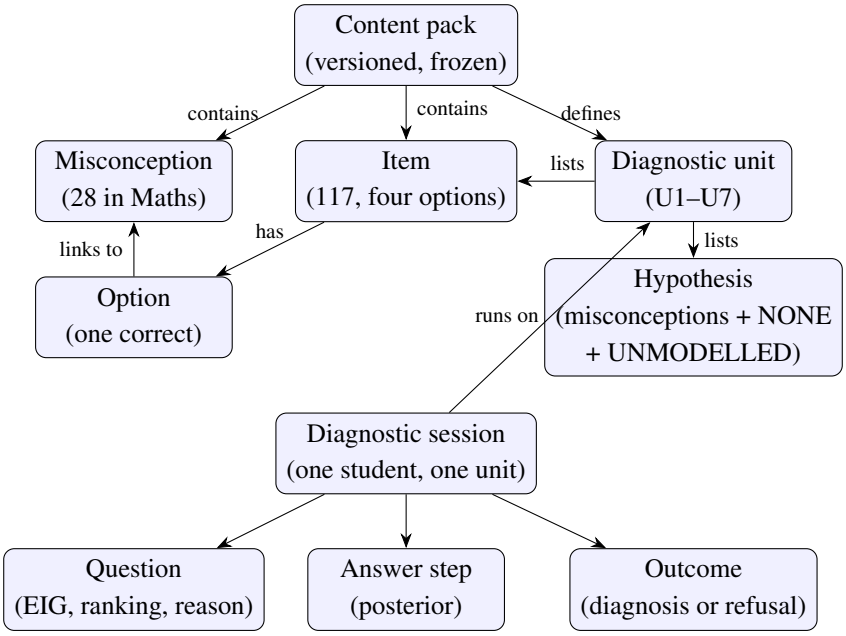



Figure 5.2: Domain model of the diagnostic core.

## 5.7 Data Design

The Iteration 1 schema (revision 3) has 41 tables, 245 columns, 66 foreign keys and 110 triggers. The design rule is that the database *checks* what the engine writes and never calculates. The ERDs below were drawn from the SQL and DBML; entities shown only as a name (for example `item` in Figure 5.4) are stubs that point to the diagram where the table is fully defined. The complete schema is in Appendix E.

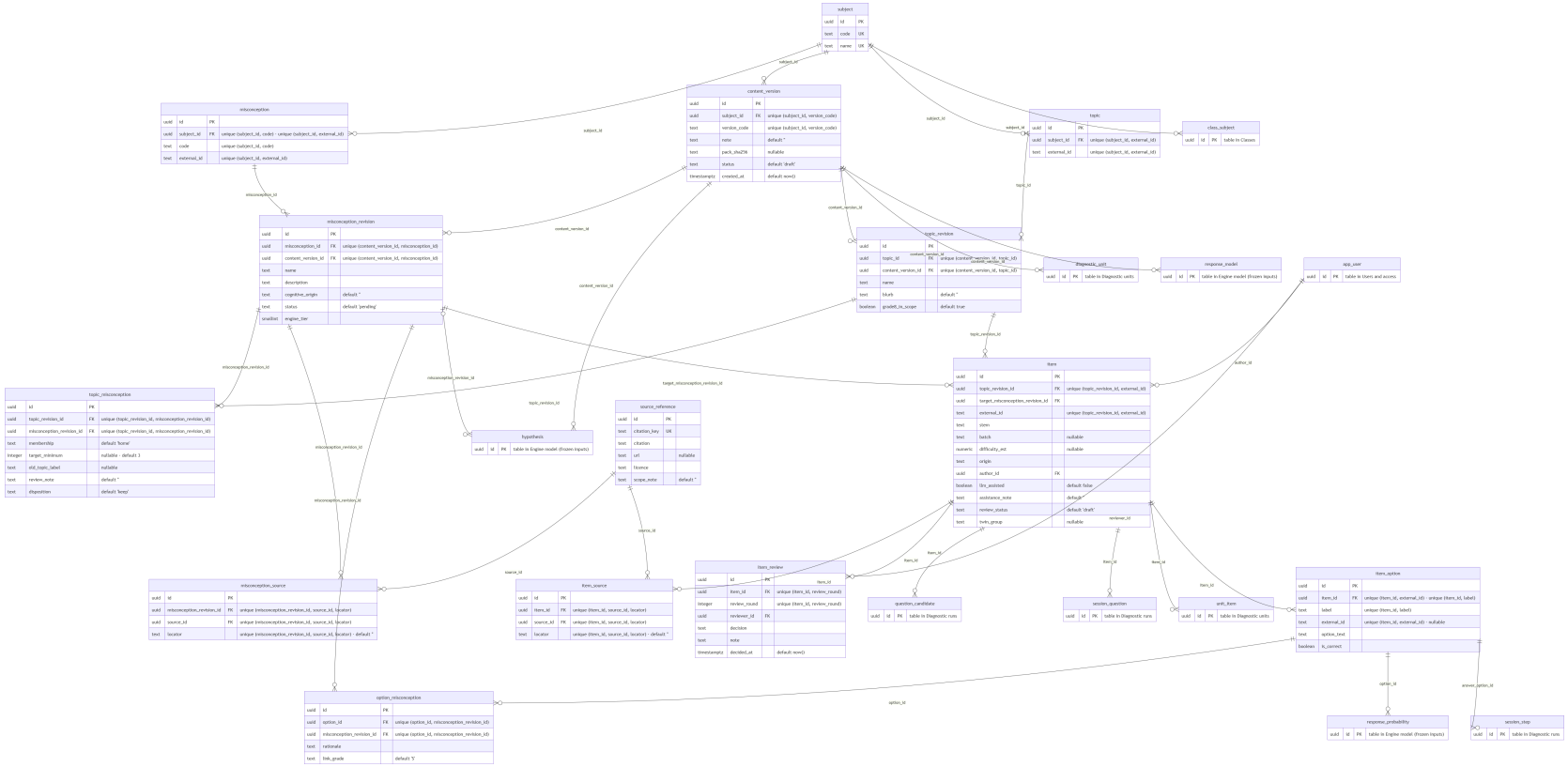



Figure 5.3: ERD: Mathematics content pack (topics, misconceptions, items, options, Q-matrix links, sources, reviews).

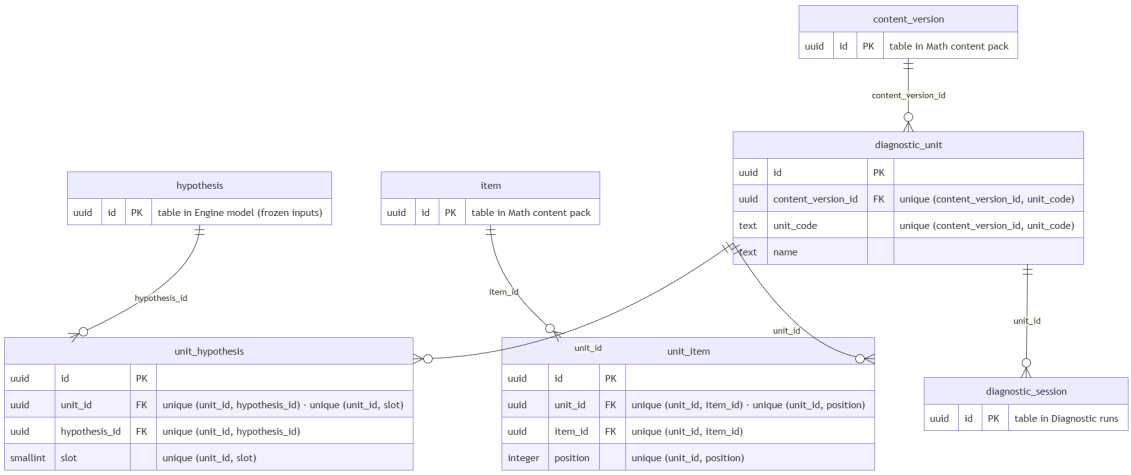



Figure 5.4: ERD: diagnostic units (ordered hypothesis slots and item positions).

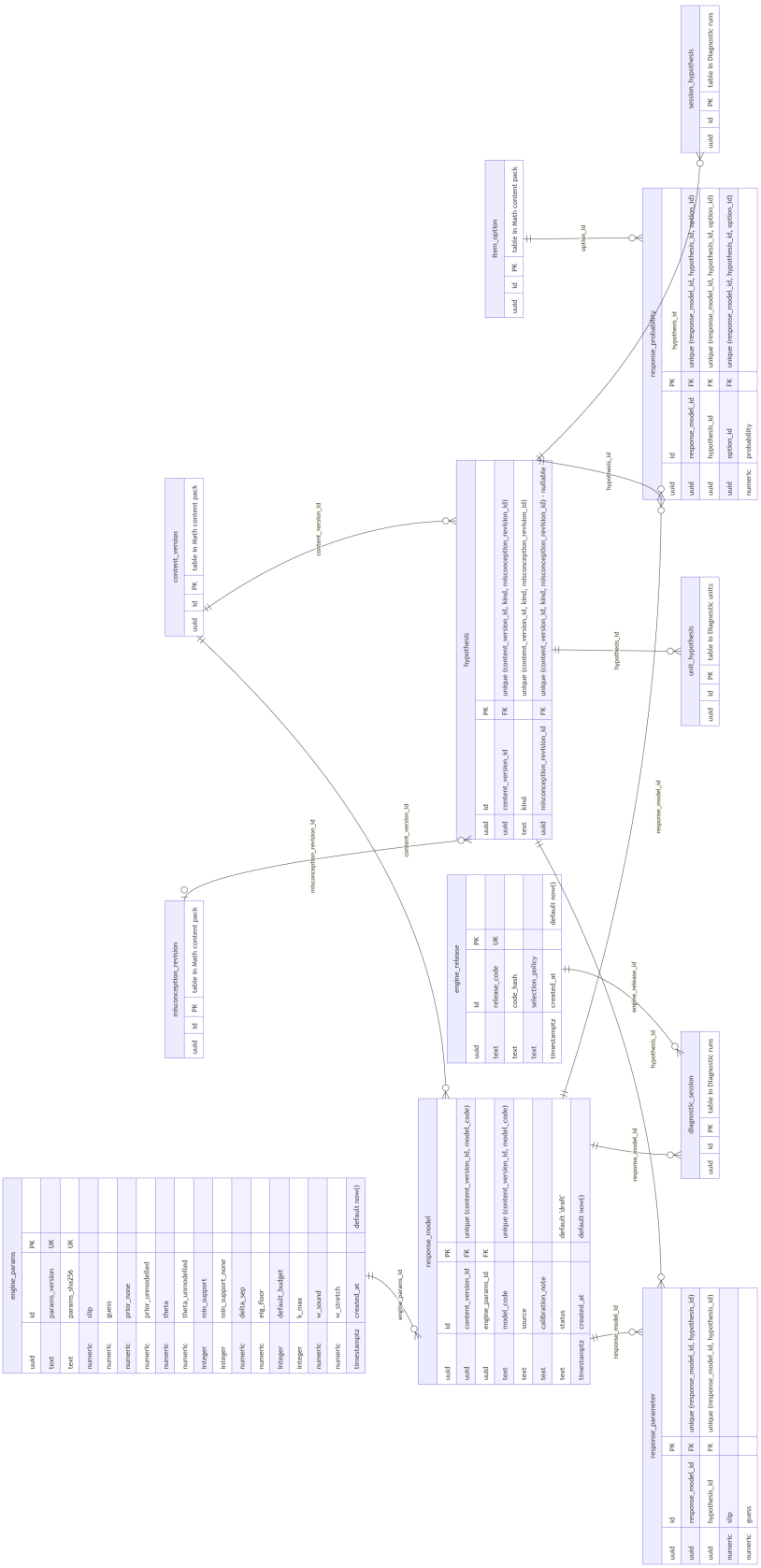



Figure 5.5: ERD: engine model and frozen inputs (parameters, response model, probabilities, engine release).

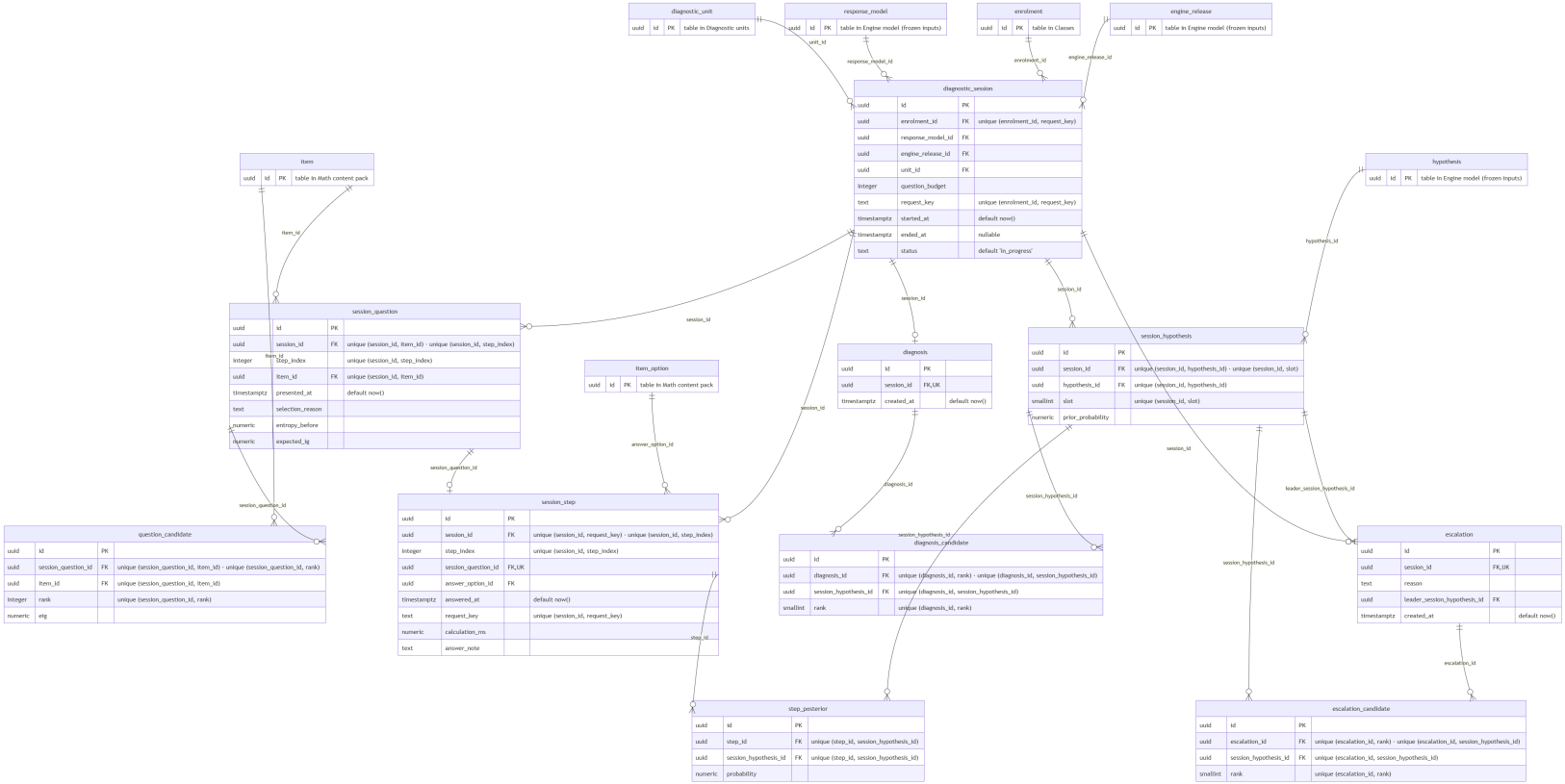



Figure 5.6: ERD: diagnostic runs (session, hypotheses, questions, candidate ranking, steps, posteriors, diagnosis, escalation).

Key integrity rules (Table 5.4) include: exactly four options and one correct option per item; the correct option has no link; approval only by a recorded review from a non-author; pack and response-model freezing with hashes and completeness checks; append-only run records; unique request keys that make retried answers harmless; and a commit-time check that every finished run has a complete, normalised trace.

Table 5.4: Selected integrity rules enforced by the database.

| Area | Rule |
|---|---|
| Items | Four options A–D, one correct (checked at commit); only draft items editable; a wrong option has at most three links |
| Review | Approve or reject only by inserting an `item_review` by an active teacher or admin who is not the author, for the next round; reviews are immutable |
| Pack freeze | Hash present; no draft or in-review item; source on each curated item; NONE and UNMODELLED exist; every unit complete; rule V5′; afterwards immutable |
| Response model freeze | Pack frozen; every approved item × hypothesis has four probabilities summing to 1; unit size $\leq k_{\max}$; rule P1 holds |
| Runs | Needs a frozen model, an enrolled student, one unit; priors written first; questions in order within the budget, never repeating an item or showing a twin; each presentation stores its ranking (ties in pack order) |
| Outcomes | A `diagnosed` run has a rank-1 misconception; an `escalated_*` run has a leader; a no-separator refusal has a tied set of at least two |

## 5.8 Process / Behavior Design

### 5.8.1 Data flow: from reviewed questions to a frozen pack

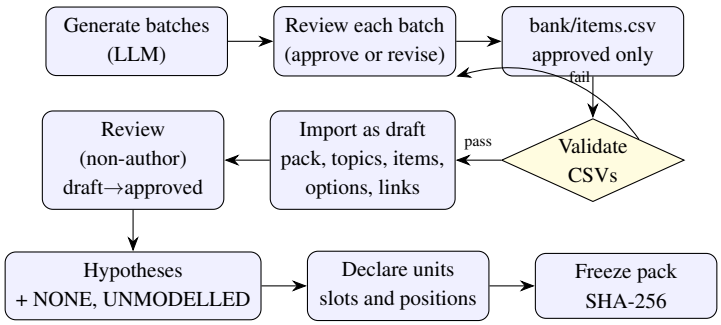



Figure 5.7: Flow 1: reviewed questions to a frozen pack. NONE, UNMODELLED and the units must exist before the freeze.

### 5.8.2 Data flow: one diagnostic step

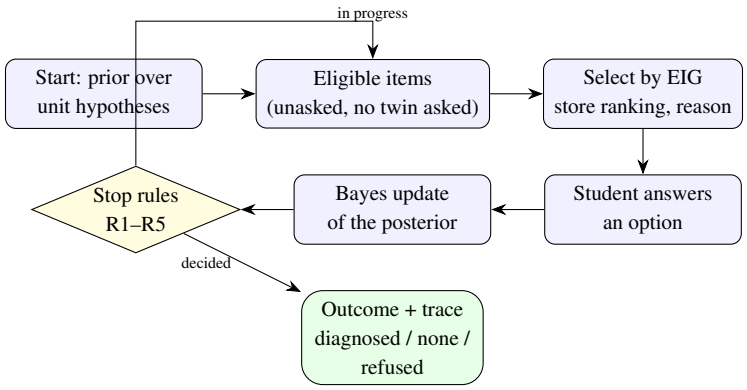



Figure 5.8: Flow 2: one answer-and-select step of a session.

### 5.8.3 State diagram: session outcomes

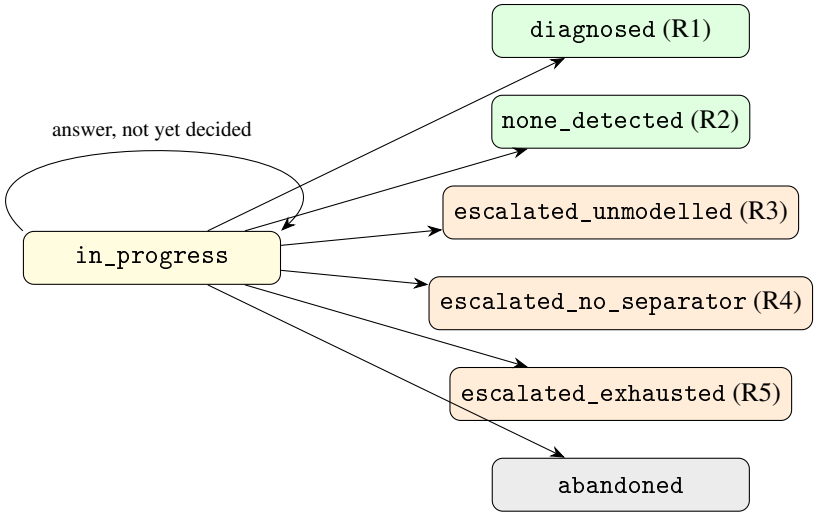



Figure 5.9: Session state machine (database column `diagnostic_session.status`). All outcomes are terminal; rules are evaluated in order R1 to R5 after each update.

### 5.8.4 State diagram: content lifecycle

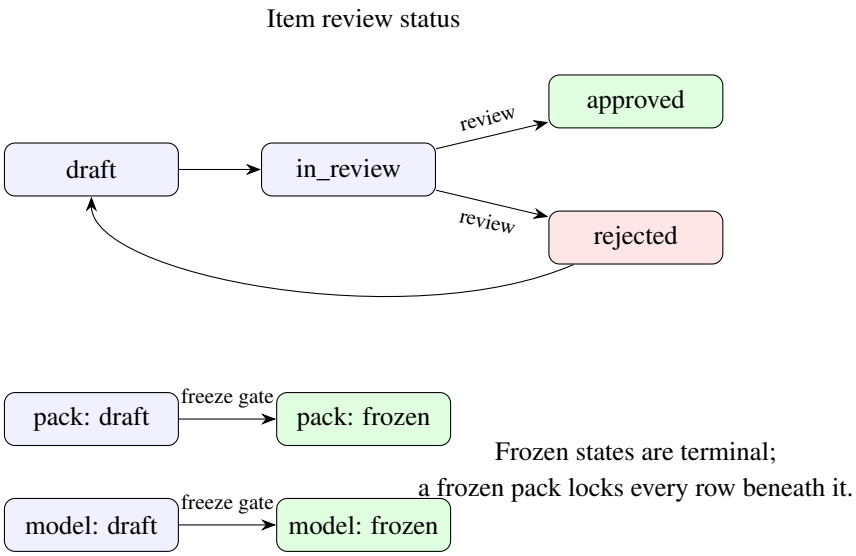



Figure 5.10: Content state machines enforced by the database (`state_guard`). Approval and rejection happen only by a review by a non-author.

# Chapter 6

# Iteration 1: Design and Implementation

## 6.1 Iteration Objectives

Build a real, tested diagnostic core for Mathematics: take an answer from the reviewed pack, update competing misconception probabilities by a stated rule, choose the next item by computed information gain, handle tie, no-separator and budget states explicitly, and leave an evidence trail. The evaluation version is the integrated `dev` branch at commit `2c266d47472e130eea2fb68ae1c19972d0994cee` (12 October 2026), the code freeze used for the live demonstration; no release tag has been created. The objective also includes a live student Mathematics diagnostic in the interface that calls the same engine.

## 6.2 Selected Use Cases / Features

UC1 (engine path and the student screen), UC2 (evidence trail, as a data structure and a terminal view), UC5 and UC6 (rules in the schema and loader), and requirements R1, R2, R3, R6, R7, R8 and R9 of Chapter 4. Teacher approval, a teacher result screen and messaging are not part of this iteration's built scope.

## 6.3 Design

The design follows the engine specification v2 (`docs/ENGINE_SPEC_DRAFT.md`), which was measured on the real bank, and the schema plan (Chapter 5). Three principles shape it: the engine is a pure function of its inputs (so every result is reproducible); the database checks writes but never calculates; and a refusal is a designed outcome with the same standing as a diagnosis.

## 6.4 Implementation

Table 6.1: Implemented modules (all under `sonder_engine/` unless stated).

| File | Content | Author |
|---|---|---|
| `model.py` | `Item`, `Params` (all assumed parameters, validated), hypothesis-list checks | Ashher |
| `bayes.py` | `prior`, `likelihood_matrix` (with rule P1 and link grades), `update_posterior` | Ashher |
| `selection.py` | `eligible_items`, `entropy_bits`, `expected_information_gain`, `select_next` | Harris (v2 alignment: Ashher) |
| `pack.py` | Builds units from rows; rules V1–V11; warnings W1–W6; exclusions | Ahmed |
| `stopping.py` | Rules R1–R5, support, separability, `check_stop` | Ahmed |
| `session.py` | `start`, `next_question`, `answer`, `replay`; immutable session state | Ahmed |
| `trace.py` | Evidence trail with field names of the schema; JSON round trip | Ahmed |
| `sonder_pack/` | The only code that reads files; unit table (JSON); `run_session` demonstration command | Ahmed |
| `sonder_api/` | Local FastAPI engine service: start, answer and trace routes; student view that hides every result; signed run cookie, shared token, origin check; in-memory run store | Ahmed |
| `Sonder-UI-master/` (`app/api`, `lib/diagnostic`, `app/student`) | Same-origin routes and the student Mathematics diagnostic screens (topic list, questions, completion, resume, error handling) | Ahmed |

## 6.5 Completed Features / Module

- Prior over a unit's hypotheses, Bayesian update, graded links (sound 1.0, stretch 0.5).
- Expected-information-gain selection with deterministic ties, full ranking and a generated reason.
- Five stopping and refusal rules and six outcomes, including the tied set of a refusal.
- A run loop that is an immutable value, replayable from the option sequence.

- An evidence trail whose fields follow the database tables.
- A loader that builds seven diagnostic units from the Maths_v2 files, applies hard rules and reports warnings and every exclusion.
- A local engine service and a live student diagnostic in the interface: the student answers real questions, can resume after a refresh, and sees only a neutral message at the end ("Your check is finished. A teacher will review it.").
- The reviewed Maths_v2 pack and its validator; the 41-table schema with automated checks in CI.

## 6.6 Technical Implementation Details

### 6.6.1 Setup and execution (to reproduce)

```
git clone https://github.com/sonder-fyp/SONDER.git
cd SONDER && git checkout dev          # or the evaluation tag
python -m pip install -e .              # Python >= 3.11, numpy >= 2.0
python -m unittest discover -s tests    # 186 tests
python -m validation.math_dataset       # bank structure check
python -m sonder_pack.maths_v2          # units, hashes, warnings
python -m sonder_pack.run_session --unit U1 --answers D,B,B,A --table
```

The live demonstration runs the same commit as two loopback processes: the engine service (`python -m sonder_api -host 127.0.0.1 -port 8000 -trace`, which needs the optional API dependencies) and the interface (`npm run dev` in `Sonder-UI-master`), opened at `http://127.0.0.1:3000`; the setup is in `sonder_api/README.md`.

### 6.6.2 Calculation

Let $\mathcal{H} = (m_1, \ldots, m_k, \text{NONE}, \text{UNMODELLED})$ with $k \leq 8$. For an item $i$ with four options and a hypothesis $h$, let $P_h(i)$ be the set of options $h$ predicts: the options linked to $h$ if any; otherwise the correct option; for NONE the correct option; for UNMODELLED all four. With $e = \text{slip}$ if $P_h(i)$ is the single correct option and $e = \text{guess}$ otherwise,

$$P(o \mid h, i) = \begin{cases} (1-e)/|P_h(i)| & o \in P_h(i) \\ e/(4-|P_h(i)|) & \text{otherwise} \end{cases}, \qquad P(o \mid \text{UNMODELLED}, i) = \tfrac{1}{4}.$$

Rows sum to one and every entry is positive. A stretch link with weight $w$ gives the row $w \cdot \text{row}_{\text{linked}} + (1-w) \cdot \text{row}_{\text{NONE}}$. The prior is $p_0(\text{NONE}) = 0.25$, $p_0(\text{UNMODELLED}) = 0.05$ and the remainder is split equally. After observing option $o$ on item $i$,

$$p'(h) = \frac{p(h)\,P(o \mid h,i)}{\sum_g p(g)\,P(o \mid g,i)},$$

and impossible evidence raises `zero_evidence`.

### 6.6.3 Selection

The information gain of an eligible item $i$, in bits, is the mutual information between the hypothesis and the answer:

$$\text{EIG}(i) = \sum_h \sum_o p(h)\,P(o \mid h,i) \log_2 \frac{P(o \mid h,i)}{P(o)}, \qquad P(o) = \sum_h p(h) P(o \mid h,i).$$

Eligible items are the unit's items minus those already asked and minus the twins of asked items. Each EIG is rounded to nine decimals; the highest wins, and an exact tie goes to the earliest item in pack order. Many real items tie exactly (T1-005, T1-006 and T1-007 all give 0.7887 bits at the start of unit U1), so the tie rule decides the demonstrated session and is part of the specification. A value in $[-10^{-12}, 0)$ is clamped to zero; anything more negative raises `negative_eig` as a defect. An empty list returns `None`.

The *reason* stored with each choice is generated from the computation, never written freely: it names the first pair among the three most probable hypotheses (excluding UNMODELLED) whose answer distributions differ by total variation greater than $\delta = 0.05$, states what each predicts, the EIG, the number of candidates, the next-best item and whether a tie was broken by item order. An unlinked item still has positive EIG while UNMODELLED has mass, because UNMODELLED answers uniformly and every other hypothesis predicts the correct answer.

### 6.6.4 Stopping and refusal

**Support** of a hypothesis set is the number of answers that its members trigger: a misconception's linked options, NONE's correct option, UNMODELLED's unlinked wrong options. Two hypotheses are separable by an item if their answer distributions differ by total variation $> \delta$. The rules are evaluated in order after each update (Table 6.2).

Table 6.2: Stopping and refusal rules (defaults: $\theta = 0.80$, $\theta_U = 0.50$, `min_support`=2, `min_support_none`=3, $\eta = 0.01$ bits, budget 8).

| Rule | Outcome | Fires when |
|---|---|---|
| R1 | `diagnosed` | The leading misconception has $p \geq \theta$ and support $\geq$ `min_support` |
| R2 | `none_detected` | $p$(NONE) $\geq \theta$ and support $\geq$ `min_support_none` |
| R3 | `escalated_unmodelled` | $p$(UNMODELLED) $\geq \theta_U$ with enough support |
| R4 | `escalated_no_separator` | Unused eligible items exist, but the set $C$ of hypotheses that no remaining item separates from the leader $\ell$ has at least two members, $p(C) \geq \theta$, $p(\ell)/p(C) < \theta$ and enough support: the leader's best possible share is below $\theta$, so the honest outcome names the tied set |
| R5 | `escalated_exhausted` | Budget used, no items left, or best EIG $< \eta$ |

### 6.6.5 Pack loading

The loader builds unit U1–U7 from `items.csv`, `topics.csv` and a hand-written unit table. Hard rules V1–V11 refuse to load: four options, one correct, unique IDs, no link on the correct option, every linked misconception inside the unit (V5′), at most eight hypotheses, at least a budget's worth of item groups, a supported pack format, and rule P1. Warnings W1–W6 are reported on every load (inseparable pairs, weak separability, hypotheses with fewer than three trigger groups, items with two unlinked wrong options, skipped items). Every exclusion is configuration, never a change to the data. The three source files are hashed (SHA-256) and the hashes, engine version and all parameters are recorded in each trace.

## 6.7 Algorithms and Technical Decisions

Table 6.3: Technical decisions and open items (spec decision IDs).

| ID | Decision | Status |
|---|---|---|
| D-1 | Merge the inseparable misconception pairs (1514/1773, 1074/1651) in the catalogue instead of keeping them for a refusal demonstration | Recommended; not decided |
| D-2 | Three items that link one misconception to three options are skipped for unit U6 at load; `guess` stays 0.30 | Applied (11 Oct.); content fix open |
| D-3 | Raise `min_support` to 3 (halves random-clicker diagnoses) | Recommended; default kept at 2 |
| D-4 | Three items belong to no unit | Left unplaced |
| D-5 | Keep companion hypotheses with fewer than three trigger groups | Kept; documented |
| Rounding | Round EIG to nine decimals before comparing; ties by pack order | Implemented |
| Weights | Stretch links weigh 0.5 until a reviewed grade column exists | Assumed |

## 6.8 Tools, Frameworks and Technologies

Table 6.4: Tools and technologies.

| Tool | Use |
|---|---|
| Python 3.11+ (CI 3.12), NumPy 2.x, `unittest` | Engine, tests |
| PostgreSQL 16 (Neon), SQL, DBML, dbdiagram | Schema and its tests |
| Mermaid, draw.io | ERD and data-flow diagrams |
| Next.js, React, Node | Scripted interface prototype |
| GitHub, GitHub Actions | Repository, pull requests, CI (interface, dataset, engine) |
| Claude (Anthropic), Claude Code | Item drafting; code, tests, diagrams (Section 8.7) |

## 6.9 Demonstration Evidence

### 6.9.1 Normal case: worked example D1 (unit U1, MATH-1379 student)

This command replays four answers; the engine chose every question:

```
python -m sonder_pack.run_session --unit U1 --answers D,B,B,A --table
```

Table 6.5: Worked example D1 on unit U1 (rounding). Top posterior entries after each answer.

| Step | Item | EIG (bits) | Answer | Posterior (top three) after the answer |
|---|---|---|---|---|
| 0 | T1-005 | 0.7887 | D, correct | NONE 0.503, MATH-1988 0.352, MATH-1248 0.039 |
| 1 | T1-006 | 0.6472 | B, correct | NONE 0.842, MATH-1379 0.066, MATH-1988 0.066 |
| 2 | T1-007 | 0.2232 | B, linked to MATH-1379 | MATH-1379 0.567, NONE 0.347, UNMODELLED 0.040 |
| 3 | T1-009 | 0.5546 | A, linked to MATH-1379 | MATH-1379 0.942, NONE 0.028, UNMODELLED 0.024 |

After step 3 rule R1 fires: `diagnosed`, MATH-1379, $p = 0.942$ with two supporting answers. Each step prints its generated reason, for example at step 2: "Chosen to separate NONE (p=0.842) from MATH-1379 (p=0.065): NONE predicts C = correct; MATH-1379 predicts B. Expected information gain 0.223 bits, best of 18 candidate items; next best T1-008 at 0.223 bits (tie broken by item order)." The first two answers were correct, so the posterior first moved towards NONE; the two answers linked to MATH-1379 then reversed it. This is why `none_detected` requires three supporting answers: with two, the session would have ended there as "no misconception".

### 6.9.2 Edge case: worked example D2 (unit U4, inseparable pair)

A student who always chooses the option linked to MATH-1514 reaches a posterior of about 0.481 for each of MATH-1514 and MATH-1773 after two answers. Both misconceptions predict the same options on every item of unit U4, so no remaining item separates them and the ceiling for the leader is below $\theta$. Rule R4 ends the session as `escalated_no_separator`, naming both. The behaviour is checked by `tests/test_`

`engine_stopping.py` (`test_d2_1514_refused_naming_both`) and `tests/test_engine_session.py`. The refusal is the designed behaviour; the underlying cause is a defect of the data (KF-06) and is not hidden by it.

### 6.9.3 The same examples in the browser

In the interface a student chooses Student, Start a diagnostic, Mathematics and unit U1, then answers D, B, B, A: the session ends after four answers and the student sees only the neutral completion message. The diagnosis (MATH-1379 at 0.9423, rule R1) is printed by the engine service's trace and by `python -m sonder_api.inspect_session`. The same flow on unit U4 with answers D, A ends after two answers with the refusal naming MATH-1514 and MATH-1773 (0.4813 each, rule R4), again hidden from the student. Both runs are checked by `scripts/check_engine_ui.py`.

### 6.9.4 Failure cases

An option outside the item raises `invalid_option`; evidence with zero probability raises `zero_evidence`; an item set that violates P1 at the chosen `guess` is refused when the unit is loaded or the session started. In each case the session state is unchanged (`tests/test_engine_session.py`). If the engine service stops during a session, the interface says it could not reach the check and offers Retry, shows no invented result, and a retry resends the identical answer only if it was not recorded, so no answer is counted twice; after a restart the in-memory run is gone and a fresh check works (interface tests).

## 6.10 Iteration Outcome

The Iteration 1 objective is met for the engine and for the student flow, with limits. The calculation, stopping and refusal rules, run loop and evidence trail are implemented and tested (merged 11 October 2026), and a student can take a real Mathematics diagnostic in the interface (merged 12 October 2026). Not achieved: saving live runs to the database (runs are held in memory and lost on restart), the production import of the pack into the hosted database, a teacher result screen, and a real login. The evidence trail is seen in the engine terminal and with `python -m sonder_api.inspect_session`, not yet on a teacher screen.

# Chapter 7

# Testing and Early Validation

## 7.1 Testing Strategy

The strategy is to make the engine's behaviour checkable against something independent of the code that produces it. Four kinds of reference are used: hand-computed values; exact rational arithmetic (Python `Fraction` and high-precision decimals) against the floating-point code; a brute-force implementation that shares no code with the module; and the worked examples of the engine specification, replayed on the real bank. Tests were written before the code for the selection module (red–green), and deliberate bugs were injected to check that the tests can fail (mutation testing). Because no real-student data exist, validation of accuracy is limited to synthetic students, and this is stated wherever such results appear.

## 7.2 Test Levels and Techniques

Table 7.1: Test levels.

| Level | Technique | Where |
|---|---|---|
| Unit | Hand-derived values, boundary cases, invalid input | `test_bayes.py`, `test_selection.py`, `test_engine_stopping.py` |
| Independent reference | Exact fractions; brute-force ranking; shuffled order | `test_bayes_math_candidate.py`, `test_selection.py` |
| Data / contract | Real-bank checks; rules V1–V11 and warnings | `test_math_dataset.py`, `test_engine_pack.py` |
| Integration (library) | Run loop, replay, trace field names against the schema | `test_engine_session.py`, `test_engine_trace.py` |
| System (synthetic students) | 43 deterministic students through the whole session; known-fault evidence | `test_engine_students.py` |
| Static / constraint | Engine imports only NumPy and pure standard-library modules; no file, network, clock or random access | `test_engine_no_ai.py` |
| Database | 75 invalid writes that must be rejected | `docs/db/iteration1/tests/behaviour_test.sql` |
| Mutation | Seven injected bugs in the selector | Manual, results below |

## 7.3 Test Cases

Table 7.2: Test inventory on the evaluation branch (186 Python tests; the interface has a further 123).

| File | Tests | Covers |
|---|---|---|
| `test_bayes.py` | 19 | Prior, likelihood, posterior by hand; invalid shapes, zero evidence; underflow guard |
| `test_bayes_math_candidate.py` | 6 | Real T1 items against exact fractions over 80 answer paths |
| `test_selection.py` | 40 | Entropy, EIG, ties, reason text, eligibility, specification examples D1 and D2, 405 exact EIG values, 300 random posteriors against brute force |
| `test_engine_pack.py` | 40 | Units U1–U7, rules V1–V11, warnings, decision reversals |
| `test_engine_stopping.py` | 22 | Rules R1–R5, order, separability, deterministic students, sensitivity |
| `test_engine_session.py` | 12 | Worked examples D1 and D2 through the run loop, twins, budget, rejected answers, replay |
| `test_engine_trace.py` | 5 | Trace fits the schema; JSON round trip |
| `test_engine_students.py` | 5 | 43-student table; known faults KF-02, 03, 07, 12 |
| `test_engine_no_ai.py` | 5 | No model, network, file, clock or random access in the engine |
| `test_math_dataset.py` | 16 | Bank structure, batches, contract |
| `test_engine_api.py` | 16 | Engine service: the student view carries no result, retries return the original reply, run ownership and origin checks |
| Interface tests (`npm test`) | 123 | Student diagnostic screens, routes, resume, retry and error handling |

## 7.4 Test Results

All 186 Python tests pass on commit `2c266d4` (`python -m unittest discover -s tests`, about 80 s, Python 3.14 locally; CI runs 3.12) and the 123 interface tests pass in CI. The database job runs the schema checks on PostgreSQL 16 in a throwaway container: 75 invalid writes are rejected, 15 of 15 checks pass, all 117 items and 468 options load, the pack and the response models at guess 0.30 and 0.24 freeze, and the database's units equal the engine's units (CI run 38175264950, 12 October 2026). The selector's seven injected bugs (wrong logarithm base, no rounding, reversed tie rule, worst item chosen, UNMODELLED as a leader, >= instead of > in the separation test, wrong guess parameter) were each caught by at least one test. The synthetic-student table of the specification

is reproduced by the run loop. The specification lists 44 students (one for each hypothesis of each unit, including NONE); after decision D-2 removes MATH-1416 from unit U6 there are 43, and the repository test asserts the counts below:

Table 7.3: Deterministic synthetic students (specification §6.1, recomputed for 43 students). Students always apply the misconception when triggered and never slip.

| Outcome | Students | Which |
|---|---|---|
| Diagnosed correctly | 26 | Most misconceptions in U1, U2, U5 and several in U3, U4, U6, U7 |
| `none_detected` (correct for NONE) | 7 | One per unit |
| `escalated_no_separator` | 8 | MATH-1074/1651 (U3, U6); MATH-1514/1773 (U4, U7) |
| `escalated_exhausted` | 2 | MATH-2093 in U4; MATH-2359 in U6 (too few trigger items) |
| Diagnosed as a different misconception | 0 | |

Decisions take two to five answers. These are results on synthetic students that apply the model's own assumptions, so they show that the machinery behaves as specified, not that real students will be diagnosed this way.

### 7.4.1 Performance

One *step* is `next_question` plus `answer` on the integrated run loop, including the stopping decision and the trace record. Timed on 12 October 2026 over 200 repetitions of four steps per unit (800 steps) on a developer laptop, excluding the database, the network and the browser:

Table 7.4: Latency of one step (target: under 300 ms).

| Unit | Items | Median (ms) | 95th pct (ms) | Max (ms) |
|---|---|---|---|---|
| U1 | 20 | 11.7 | 22.8 | 54.3 |
| U4 | 16 | 10.1 | 23.5 | 95.3 |
| U7 | 16 | 9.9 | 23.6 | 51.7 |

An earlier run on the same engine code gave medians of 7.0 to 8.5 ms, and the selection step alone (`select_next` with an update) a median of 2.6 ms, so the figures vary with machine load. One machine and small units: they show that the target is met by more than an order of magnitude, not that the deployed system meets it.

## 7.5 Acceptance Testing / Validation

Acceptance of the criteria of Table 4.7: R1, R2 and R8 pass for the engine, R9 passes for the student flow (local checkpoint), R6 passes for its rules but not for a production import, R7 passes; R3 passes only for the half that can be tested (no subject branching in code; no second subject pack exists); R4 and R5 are deferred. Validation against real students, teacher agreement ($\kappa$) and calibration cannot be done before a pilot.

## 7.6 Edge and Failure Cases

Table 7.5: Edge and failure cases.

| Case | Condition | Result |
|---|---|---|
| Inseparable pair | Unit U4, student always choosing the MATH-1514-linked option | Refusal naming MATH-1514 and MATH-1773 (R4); posterior ≈0.481 each |
| Invalid option | Option outside the item | `invalid_option`; state unchanged |
| Impossible evidence | Zero-probability observation | `zero_evidence`; not an outcome |
| Violating model | Three-option link at `guess` 0.30 | Item skipped at load (D-2); the model is refused if forced |
| Empty candidate list | No eligible items | `select_next` returns `None`; R5 ends the session |
| Tie of EIG | Exactly equal values | Earliest item in pack order; reason says a tie was broken |
| Random clicker | A student always picking option A | Diagnosed in six of seven units after two to five answers (KF-03) |
| Service stops | The engine service stops during a student's session | The interface says it could not reach the check and offers Retry; no result is shown; a retry is not counted twice |

## 7.7 Defects and Corrective Actions

Table 7.6: Defects found during the iteration and their handling.

| Defect | Corrective action |
|---|---|
| First selector draft diverged from the specification (raised an error instead of returning `None`, no negative-EIG guard, no ranking, wrong reason rule) | Rewritten test-first against the specification; the review question "does it implement everything the spec decided?" found it |
| Early specification described an 80-item bank and said no misconception links more than one option; measured bank has 117 items and three three-link items | Specification v2 re-measured on the real bank |
| First data-flow diagram created hypotheses after the freeze, which the database refuses | Diagram and document corrected |
| Three of four first ERD images had import defects (stub entities, stray text) | Re-exported from corrected diagram code |
| Four defects in the first engine-to-interface connection (a finished run blocked new ones, localhost was rejected, misleading error messages, Back restarted the check) | Found in an independent review and fixed before merging, with tests |
| The AI claimed the dataset had no diagnostic units, and suggested editing the dataset | Corrected from the specification; the dataset is never changed, problems are handled in the loader and recorded as known faults |
| A boundary test for the separation threshold was flaky (floating-point total variation near 0.6) | Replaced by a test using the exactly computed total variation |
| Claim of 16 batches in the bank (15 are in it; one is a draft) | Corrected in the manifest and documents |
| Five forced misconception links in the bank | Fixed on 10 October 2026 (batches B13 and B08) |

## 7.8 Current Validation Status

The calculation is verified against independent references and specification examples; the stopping and refusal behaviour is verified on constructed and synthetic cases; the content pack loads under the stated rules; the database rules are exercised by 75 negative cases and 15 of 15 checks in CI; and the student flow is checked by 16 service tests, 123 interface tests and an end-to-end HTTP script. Not validated: any behaviour on real students; the production import into the hosted database and the saving of live runs; usability and a browser walkthrough by an automated test (it is checked by hand); and security beyond

the service token, signed run cookie and origin check, and the absence of secrets and I/O in the engine. The 14 known faults (Table 7.7) list what is wrong today.

Table 7.7: Known faults of the engine and its data (full list with evidence: `sonder_engine/KNOWN_FAULTS.md`).

| ID | Fault | Decision |
|---|---|---|
| KF-01 | Three U6 items link one misconception to three options; skipped at load | D-2 |
| KF-02 | MATH-1416 dropped from U6 as a consequence | D-2 |
| KF-03 | Random answering is diagnosed often at `min_support=2` (about 40% pooled in the specification) | D-3 (open) |
| KF-04 | Companion hypotheses with fewer than three trigger groups cannot be diagnosed in their unit | D-5 |
| KF-05 | Three items belong to no unit | D-4 |
| KF-06 | MATH-1514/1773 and MATH-1074/1651 are inseparable | D-1 (open) |
| KF-07 | MATH-1965 is a subset of MATH-2039; partial 2039 holders are diagnosed as 1965 | D-1 (open) |
| KF-08 | Weakly separable pairs | None |
| KF-09 | No link grades; every link counts as sound | T-1 |
| KF-10 | Every parameter assumed; data synthetic | None |
| KF-11 | Thresholds $\theta \neq 0.80$ untested | D-6 |
| KF-12 | Partial holders often look like NONE | None |
| KF-13 | Unlinked wrong options push towards UNMODELLED | None |
| KF-14 | Reasons quote option letters, not text | None |

# Chapter 8

# Project Management and Development Evidence

## 8.1 Work Completed

- Repository, branch and pull-request workflow, contribution rules and five CI jobs (interface build and tests; dataset validator; engine tests; database schema checks in a throwaway PostgreSQL container; engine API tests).
- Maths_v2 content pack: 117 reviewed items, 468 options, 28 misconceptions in seven topics; dataset validator and an independent review of the first topic (T1) by Ashher.
- Engine specification v2, re-measured on the real bank, with decisions D-1 to D-6 and tasks T-1 to T-3, and its amendments (D-2 to D-5).
- The engine library: prior, likelihood, posterior, link grades and twin filtering; EIG selection; stopping and refusal rules; run loop; evidence trail; pack loader; command-line replay; 186 Python tests in total; 14 documented known faults.
- A local engine service and a live student Mathematics diagnostic in the interface, with 16 service tests, 123 interface tests and an end-to-end check script.
- PostgreSQL schema revision 3 (41 tables, 110 triggers) with a behaviour test suite and a data-load probe, both run automatically in CI.
- Design artefacts: seven ERDs, the data-flow document, the architecture diagram, state diagrams, and a proposed class-flow plan.

## 8.2 Work Remaining

Saving live runs and their evidence trail to the hosted database and the production import of the pack (T05.07); a teacher result view and a real login; resolution of the open specification decisions (D-1, D-3, T-1, T-2); the noisy-student and random-clicker simulations as committed, reproducible tests; a reviewed link-grade column; a bank-wide independent human spot-check of the item bank (Ashher reviewed T1 only); a second content pack (Physics or Chemistry) to test requirement R3; the tagged evaluation commit. Later iterations are in Chapter 9.

## 8.3 Individual Responsibilities

See Table 3.3. Each work item has a lead and a reviewer; the work division is recorded in `ITERATION_1_TEAM_WORK_DIVISION_MATH_ONLY.md`: Harris reviews Ahmed's pull requests, Ahmed reviews Ashher's, and Ashher reviews Harris's.

## 8.4 Contribution of Each Team Member

Table 8.1: Contribution by member (pull requests merged into `dev`, counted by branch owner; #3 is a `dev` to `main` synchronisation).

| Member | Merged PRs | Pull requests | Substantive contribution |
|---|---|---|---|
| Muhammad Harris Azhar | 5 | #5, #11, #14, #20, #26 | Maths_v2 reviewed pack and its review notes; engine specification v2; EIG selection and its tests; ERDs and data-flow design |
| Syed Ashher Majid | 8 | #6–#9, #22–#25 | Dataset validator and the independent T1 item review; prior, likelihood and posterior; spec v2 contract, per-item P1 check, link grades, twin filtering and validator alignment |
| Ahmed Laiq | 21 | #1, #2, #4, #15–#19, #21, #27–#38 | Repository, CI and rules; specification draft and amendments; schema revisions 1–3, tests and CI checks; architecture diagram; pack loader, stopping rules, run loop, trace, synthetic-student harness, known-fault register; engine service and live student screen; class-flow plan |

The panel's first comment applies: the commit counts on `dev` (Ahmed 52, Ashher 31, Harris 11 at commit 2c266d4, merges included) and the merged pull-request counts are uneven. Commit counts are not used here as evidence of contribution; the table lists what each member designed, implemented, tested or reviewed. The individual evidence sections of the supervisor's worksheet carry each member's own account.

## 8.5 Repository and Version Control Evidence

Repository: https://github.com/sonder-fyp/SONDER. Branches: `main` (demo-ready, evaluation tags), `dev` (integration), feature branches named `<name>/<feature>`. Rules (CONTRIBUTING.md): no direct pushes; a pull request into `dev` with CI green; `dev` to `main` with one approval and a regular merge commit. Branch protection is not enabled on GitHub, so these rules are conventions. Since pull request #15, merges into `dev` need no GitHub approval; each pull request is nevertheless reviewed by the reviewer the work division names. The repository was created on 9 October 2026 from existing proposal documents and the proof-of-concept interface. Evaluation commit: `2c266d47472e130eea2fb68ae1c19972d0994cee` on `dev`, 12 October 2026 (code freeze, used for the live demonstration); no release tag exists yet. No task board or GitHub issues are used: work items follow the task IDs T00 to T14 of the work-division document, and the pull requests are the record of completed work. Latest CI run on `dev`: https://github.com/sonder-fyp/SONDER/actions/runs/38177574282 (success).

## 8.6 Milestones and Commit Evidence

Table 8.2: Milestones (merge dates from the repository history).

| Date | Pull request | Milestone |
|---|---|---|
| 10 Oct 2026 | #1–#5 | CI fixes; engine specification draft; Maths_v2 pack merged into `dev` |
| 11 Oct 2026 | #6–#9 | Dataset validator; Bayes prior, likelihood, posterior |
| 11 Oct 2026 | #11, #14 | First ERD and data flow; engine specification v2 |
| 11 Oct 2026 | #15–#19 | Contribution rules; schema revisions 2 and 3; SQL and end-to-end schema design |
| 11 Oct 2026 | #20 | EIG selection with tests |
| 11 Oct 2026 | #21–#26 | Architecture diagram; engine v2 contract, link grades, eligibility, validator; revision-3 ERDs and data flow |
| 11 Oct 2026 | #27–#34 | Spec amendments; pack loader; stopping rules; run loop and trace; synthetic-student harness; database checks in CI |
| 12 Oct 2026 | #35–#38 | Engine service and live student screen; architecture diagram update; Word lock file removed; class-flow plan |

Twenty-six of the 35 pull requests merged into `dev` were merged on 11 October and four more on 12 October. This reflects how integration was done late, not how the work was spread; it is recorded as a deviation in Chapter 3. Ahmed's engine work was delivered as a planned sequence of small pull requests, each building on the last and each listing its tests. The pull-request list and the CI runs are available at the repository's `pulls` and `actions` pages (Appendix D).

## 8.7 Tools and Tool-Assisted Development

Generative AI levels follow the course definition: L1 supportive, L2 assisted, L3 core assist, L4 substituted. No artefact is presented as L4.

Table 8.3: GenAI register for materially assisted artefacts.

| Artefact | Tool and level | What entered the project | Verification | Owner |
|---|---|---|---|---|
| Maths_v2 item bank | Claude; L2 | Items, options and misconception links generated in batches; only approved batches kept; one batch left as a draft | Review of every batch; link audit; validator; independent T1 review; five forced links fixed | Harris |

| Artefact | Tool and level | What entered the project | Verification | Owner |
|---|---|---|---|---|
| `selection.py`, `test_ selection.py` | Claude Code; L3 | Module written test-first from the specification; all kept | Exact-arithmetic and brute-force checks; spec examples; seven injected bugs; first draft found diverging and rewritten | Harris |
| ERDs and data-flow document | Claude Code; L3 | Mermaid code and document; exported and edited in draw.io | Checked against the SQL and DBML; three defective images re-exported; wrong freeze order corrected | Harris |
| Engine specification v2 | Claude Code; L3 | Text generated from Harris's plan, reviewed and re-prompted | Worked examples reproduced by tests; open decisions listed | Harris |
| Bayesian prior, likelihood, posterior | OpenAI Codex; L3 | Code, input checks, tests and notes from the team spec | 19 core and 6 candidate-T1 tests; hand and exact-fraction checks | Ashher |
| Validator and independent T1 review | OpenAI Codex; L2 | Validator code and tests; a draft option-level review | 16 validator tests; all 20 T1 marked answers checked; 11 weak links flagged | Ashher |
| Spec v2 contract, P1 check, link grades, twin eligibility | OpenAI Codex; L2 | Model validation, link-grade blending, twin exclusion and tests | Weight-1 and twin-order tests; three P1 conflicts reported, not edited away | Ashher |
| Specification draft and amendments | Claude Code; L3 | Draft and decisions D-2 to D-5 | Rechecked against the bank and the engine | Ahmed |
| Database schema revisions, tests, tools | OpenAI Codex and Claude Code; L3 | Schema, DBML, behaviour tests, import probe, Docker test runner | 75 refused writes; 15 of 15 checks in CI; all 117 items load | Ahmed |
| Architecture diagram | Claude Code; L3 | Specification and SVG generator | Layout checks; labels checked against `dev` | Ahmed |
| Engine run loop, loader, stopping rules, trace, harness | Claude Code; L3 | Implemented from Ahmed's reviewed plan, test-first | 89 run-loop tests; D1, D2; 43 synthetic students; injected faults caught | Ahmed |
| Engine service and live student UI | OpenAI Codex (GPT Sol 6.1); L3, reviewed by Claude Code | Service and screens; problems found in an independent review were fixed | Four defects fixed; 16 service and 123 interface tests; live checks | Ahmed |
| Class-flow plan | OpenAI Codex; L2 | Four-page proposal | Checked against the class tables; labelled a proposal | Ahmed |
| This report | Claude Code; L2 | Chapter drafts from the repository, worksheet and proposal | Figures and counts checked against the repository; authors review every chapter | All |

The diagnostic calculation contains no generative model, and the engine never calls one at run time; the repository tests assert this. Mistakes in AI output that were found and corrected are listed in Table 7.6.

## 8.8 External Components and Dependencies

Table 8.4: External components, risks and fallbacks.

| Component | Purpose | Risk or limitation | Fallback |
|---|---|---|---|
| Python 3.11+, NumPy 2.x | Engine | Version drift; float rounding (guarded by tolerances and rounding rules) | Pin the version range |
| FastAPI 0.120.0, uvicorn 0.38.0, httpx 0.28.1 | Local engine service and its tests (optional extras) | Needed only for the live flow; one process holds runs in memory | The library and command-line session need none of them |
| PostgreSQL 16 (Neon); Docker image for tests | Storage; throwaway database for CI checks | Hosted-service availability and cost; schema size; Docker needed locally | Local PostgreSQL; the engine runs without it |
| Next.js, React | Interface; student diagnostic live | Other screens are scripted fixtures | Keep the labelled demo |
| Eedi catalogue (Kaggle) | Source of misconception names for Mathematics | Licence and redistribution terms not yet recorded; single-source bias | Self-authored misconceptions |
| Claude and OpenAI Codex (LLMs) | Draft items, code, diagrams | Wrong or forced links (about 90 stretch links remain); generation errors | Frozen, reviewed pack; not needed at run time |
| GitHub and Actions | Repository, CI | Availability; branch protection off | Run the same commands locally |

## 8.9 Risks and Mitigation

Table 8.5: Risk register.

| Risk | Level | Mitigation |
|---|---|---|
| Parameters are assumed and the data are synthetic, so accuracy claims fail on real students | High | Do not claim accuracy; calibrate on a pilot; report negative results |
| Uneven contribution within the team | High | Per-member evidence table; named reviewers; weekly contribution check; individual ownership in the worksheet |
| Live runs are in memory and not saved; no login or teacher view | High | Save runs and traces to the session tables, add a teacher result view and a login (Iteration 2 start) |
| Inseparable misconception pairs make refusals a data artefact | Medium | Merge pairs (D-1); write separating items (T-2) |
| Random clickers diagnosed too often | Medium | Raise `min_support` to 3 (D-3) after a team decision |
| Schema too heavy for the current scope | Medium | Open decision on slimming; the schema stays behind the engine interface |
| Panel scope concern: weak technical challenge | Medium | Challenge made explicit (Chapter 2); scope agreed with the supervisor |
| Ethics approval delays the pilot | Medium | Start the approval process in FYP-2 Month 1; simulated studies meanwhile |
| Licence of the Eedi catalogue; no repository licence file | Low–medium | Record the licence; keep only misconception names and IDs; add a repository licence |

## 8.10 Security and Privacy Considerations

No student data has been collected and none is in the repository. The proposal commits to pseudonymous identifiers, data minimisation, written guardian consent, student assent, withdrawal rights, retention limits and formal ethics approval before any pilot with minors (approval not yet obtained). The local engine service requires a shared token, owns each run through a signed HttpOnly cookie (not marked secure, since it runs on local HTTP), checks the request origin and listens on the loopback address only; there is no real login and no deployed service. The schema defines roles, hashed credentials and tokens, and append-only and frozen records; the engine performs no I/O. CONTRIBUTING.md forbids secrets in the repository (connection strings go in GitHub secrets or local environment files); no automated secret scan has been run yet. An RBAC audit and hardening are planned for Iteration 4.

## 8.11 Current Limitations

- Live runs are held in memory and lost on restart; they are not saved to the database. There is no real login and no teacher result screen; the evidence trail is read in the engine terminal.
- All probabilities and thresholds are assumed; the bank is synthetic; accuracy, calibration and teacher agreement are untested.
- Only Mathematics content exists; the second half of requirement R3 is untestable. The bank has a bank-wide independent review for topic T1 only.
- Fourteen known faults (Table 7.7); in particular inseparable pairs, skipped items and a high random-clicker diagnosis rate.
- The browser walkthrough is checked by hand; no automated browser test exists.
- One teacher interview; no student or usability evidence.
- Specification measurements are reproduced in the repository for the deterministic students only; the noisy-student and random-clicker figures rest on reference scripts that are not in the repository.

# Chapter 9

# Remaining FYP Plan

## 9.1 Remaining Scope

Everything in the approved scope (Section 1) that Iteration 1 did not deliver: integration of the engine with a service, the database and the interface; Physics and Chemistry content; teacher approval and escalation; interventions and the verification loop; the self-improving question bank; announcements and messaging; the AI consultant, learning hub and tutor chat; evaluation against baselines; deployment, hardening and a classroom pilot. Scope changes require a recorded decision with the supervisor (action 4, Chapter 3).

## 9.2 Remaining Modules and Features

Table 9.1: Remaining modules.

| Module | Remaining work |
|---|---|
| Engine service and importer | Move the local service from an in-memory store to the database: idempotent commit per answer, importer from the pack files into the hosted schema (T05.07), pinned parameters and engine release |
| Interface integration | Teacher result view with the evidence trail, real login and class scoping; replace the remaining scripted fixtures with real data |
| Teacher gate | Approve, correct, reject; escalation queue for refusals; release control for student and parent views |
| Interventions and verification | Study notes, vetted video links; fresh-question immediate check and a 14–21 day delayed check with a scheduler |
| Question-bank pipeline | Gap scan for inseparable pairs, question generator, distractor engine, quality scoring, two-reviewer queue |
| Content | Review the Mathematics bank further; Physics and Chemistry packs from literature-sourced misconceptions |
| Communication and AI | Classes, announcements, direct messaging; AI consultant (pace, escalation), tutor chat, resource finder, bounded and read-only |
| Evaluation and deployment | V1 to V6 studies; containerised deployment, 2 Mbps optimisation, security and RBAC audit; classroom pilot |

## 9.3 Remaining Iterations

Iteration 2 (months 3–4): teacher approval and escalation, interventions, the verification loop, the question-bank pipeline, the second validation pass, V1 and the first deployment attempt; gate FYP-1 Final-1 with an end-to-end core flow and baseline result. Iteration 3 (months 5–6): validation and verification complete, the question-bank loop, AI product surface, expanded classes with messaging, V2, V3, V5, V6, measured non-functional requirements; gate Mid-2. Iteration 4 (months 7–8): freeze and audit of the datasets,

deployment and hardening, the classroom pilot with delayed follow-up, final report and handover; gate Final-2.

## 9.4 Implementation Plan

Immediate order (before Final-1): (1) import the pack into the hosted database and save each run and its evidence trail, replacing the in-memory store (Ahmed, 2026-10-18); (2) a teacher result view (2026-10-25); (3) team decisions D-1 and D-2 (Harris, 2026-10-18), D-3, T-1 and T-2, and the link-grade column; (4) the noisy-student and random-clicker simulations as tests (Harris, 2026-10-20); (5) the bank-wide independent link review (Harris and Ashher, 2026-10-25); (6) teacher approval and a real login; (7) a second subject pack to exercise requirement R3. Each item is a task with an owner, a branch, a pull request and a test.

## 9.5 Testing and Validation Plan

Table 9.2: Planned studies (from the proposal).

| Study | Question |
|---|---|
| V1 | Simulation: does entropy selection improve items-to-diagnosis and error or refusal behaviour over fixed or random selection under known synthetic truth and noise? |
| V2 | Baselines: how does sequential diagnosis compare with one-shot LLM classification and retrieve-and-rerank on the same labelled examples as the catalogue grows? |
| V3 | Offline matching: how often does the engine's diagnosis agree with held-out Eedi labels? |
| V4 | Classroom pilot: can a teacher-led diagnose, intervene, immediate-verify, delayed-verify cycle run with consent, and with what outcomes? |
| V5 | Contextual validity: do teachers judge catalogue entries and wording suitable for Grade 8, with reviewer agreement? |
| V6 | Pre-pilot item check: do student choices match the Q-matrix's predicted distractor behaviour? |

Metrics: confusion matrix, precision and recall per misconception; confident-misdiagnosis rate with coverage, reported together (targets: at most 5% and at least 70%); items-to-diagnosis; refusal precision and recall; calibration (Brier score, reliability); Cohen's $\kappa \geq 0.7$ against an independent teacher; latency under 300 ms. Results are reported by subject and failure category, and negative results are reported. Each member also keeps three technical growth questions in the worksheet, for example how slip and guess should

be estimated from real answers, whether greedy one-step selection is adequate under a budget of eight, and how to set thresholds to meet both targets.

## 9.6 Deployment Plan

Containerised deployment (Docker Compose is sufficient at the pilot's scale of about 30 users): the interface and engine on one CPU host with PostgreSQL, and a GPU host only if the model service of Iteration 3 is self-hosted. Targets: a usable flow at about 2 Mbps with session recovery, support for low-end Android devices, installation by someone outside the team, a rehearsed rollback before Mid-2, and measured non-functional requirements. Hosting decision for Iteration 2: the interface and engine service run on one CPU host and the database stays on the hosted PostgreSQL service (Neon); the model class for Iteration 3 is chosen against the available hardware when that iteration starts.

## 9.7 Tentative Timeline

Table 9.3: Relative timeline (months $M1$–$M8$ from the roadmap); calendar dates to be added.

| Workstream | Months |
|---|---|
| Database save, teacher result view, login, interface integration | M1–M3 |
| Mathematics content review; Physics and Chemistry drafts to reviewed packs | M1–M6 |
| Teacher approval, escalation, interventions | M3–M4 |
| Verification loop and delayed check | M4–M6 |
| Engine evaluation V1–V3 | M3–M6 |
| Question-bank pipeline | M4–M6 |
| Learning hub, AI assistance, parent and admin views, messaging | M5–M6 |
| Integration, system tests, non-functional measurement, deployment rehearsal | M4–M7 |
| Ethics and access, pilot preparation, V5, V6 | M1–M6 |
| Pilot (V4), durability, final audit and report | M7–M8 |

## 9.8 Expected Final Deliverables

A deployed system with the four role views connected to the engine; reviewed, frozen packs for the subjects completed; the teacher-gated diagnose, intervene and verify cycle; the evidence trail and audit; the evaluation of V1 to V6 with baselines; security and RBAC audit; the final dataset freeze and audit; the final report, user and deployment documentation, and the source repository with tagged releases.

# Chapter 10

# Conclusion

Diagnostic tools that commit to one misconception per wrong answer cannot tell a student who holds a misconception from one who made an unrelated mistake. Sonder treats that ambiguity as its computing problem: keep a posterior over competing misconceptions, ask the question that most reduces the uncertainty, and refuse when the evidence is not enough. The research (Chapter 2) found no deployed system that combines these, and a teacher interview confirmed the premise.

Measuring the real item bank reshaped the design: sessions are scoped to diagnostic units, twin items are filtered, links carry grades, and a response-model rule (P1) limits how many options a misconception may claim. The requirements (Chapter 4) are measurable and traced to design, code and tests. The Iteration 1 engine (Chapter 6) is complete: posterior update, expected-information-gain selection with deterministic ties and generated reasons, five stopping and refusal rules, a replayable run loop and an evidence trail, loading a reviewed Mathematics pack into seven units. It is verified by 186 Python tests and 123 interface tests against exact arithmetic, brute-force references, the specification's worked examples and 43 deterministic synthetic students, and one step takes a median of about 10 ms against a 300 ms target. A student can take a real Mathematics diagnostic in the browser, computed by the same engine, and sees only a neutral completion message because every result stays behind the teacher gate.

The honest status is that the core is built and the product around it is not. The data are synthetic and every parameter is assumed, so the proposal's accuracy targets are not testable; fourteen known faults, including two pairs of misconceptions that the data cannot separate, are documented rather than hidden. Live runs are held in memory and are not saved to the database, there is no login and no teacher result screen, contribution within the team is uneven, and only Mathematics exists. FYP-2 will save runs and traces to the database, add the teacher gate and verification loop, extend the content to Physics and Chemistry, calibrate on real responses, deploy the system, and evaluate it against baselines.

# Appendix A

# Proposal Action Register

The register is in Chapter 3; this appendix records the panel's exact comments and the supporting evidence for each action.

Table A.1: Panel comments and evidence.

| No. | Panel comment (exact) | Supporting evidence |
|---|---|---|
| 1 | "Coordination is weak between students; everyone needs to put in effort, not just one member." | Work-division document with lead and reviewer pairs; merged pull requests by member (Table 8.1); CONTRIBUTING.md |
| 2 | "There is currently a weak technical challenge; more features with technical challenges should be included and the scope expanded." | Engine specification v2; Section 2.8; stopping and refusal rules, link grades, unit design, frozen packs (Chapter 6) |
| 3 | "The system needs further development by incorporating additional features to enhance its usefulness for both students and teachers." | Proposal Iteration Plan; roadmap; Chapter 9 |
| 4 | "Discuss scope with your supervisor." | Scope meeting with Dr. Ali Zeeshan Ijaz, 9 September 2026 (Appendix F); the agreed scope is to be written into the roadmap |
| – | "Overall, the presentation was good, the work is good, and the project idea is clearly presented." | No action required |

# Appendix B

# Detailed Requirements

Status is that of the evaluation branch `dev` (11 October 2026).

## R1 — Next-question selection (functional; owner Harris)

**Statement.** Given the current posterior over a unit's hypotheses and the eligible items, the engine chooses the item with the highest expected information gain in bits. Exact ties go to the earliest item in pack order. It returns the full ranking, the entropy before the answer and a generated reason.
**Source.** Proposal FR-01; engine specification §3.4.
**Acceptance.** EIG matches exact arithmetic to $10^{-11}$; choice and ranking match a brute-force reference on random posteriors; the specification's worked examples reproduce to four decimals (D1: T1-005 0.7887, T1-006 0.6472, T1-007 0.2232, T1-009 0.5546; D2: T4-001 0.7517, T5-011 0.3668); an empty list returns `None`; a negative EIG beyond $-10^{-12}$ raises an error; asked items and their twins are excluded.
**Evidence.** `selection.py`; 40 tests; seven injected bugs all caught.
**Status.** Met as a library. **Gaps.** Reasons show option letters, not text (KF-14); the budget and stopping decision belong to the run loop, which Ahmed built.

## R2 — Refusal as a first-class outcome (functional; owner Ahmed)

**Statement.** When a supported diagnosis cannot be made, the run ends with an explicit refusal and the evidence behind it; a forced diagnosis is never presented.
**Rules (spec §3.5).** R1 diagnosed: leader $p \geq 0.80$ with at least two supporting answers. R2 none_detected: $p(\text{NONE}) \geq 0.80$ with at least three. R3 escalated_unmodelled:

$p(\text{UNMODELLED}) \geq 0.50$. R4 escalated_no_separator: a tied set that no remaining item can separate. R5 escalated_exhausted: budget (8) spent, no items left, or best EIG below 0.01.
**Acceptance.** Each outcome is produced by a constructed case; the D2 student (MATH-1514, MATH-1773) is refused naming both; a student without a misconception is `none_detected`.
**Evidence.** `stopping.py`, `session.py`; `test_engine_stopping.py` (22), `test_engine_students.py`.
**Status.** Met in the engine and shown live in the student flow; results are not yet saved to the database or shown to a teacher. **Gaps.** Thresholds assumed (KF-10, KF-11); random clickers diagnosed often (KF-03).

## R3 — One engine for every subject (functional)

**Acceptance.** No subject-specific code; a pack of another subject loads unchanged.
**Status.** Partial: no subject branching in the code; only the Mathematics pack exists, so the second half is untestable.

## R4 — Teacher approval gate (functional)

**Statement.** No diagnosis reaches a student or parent before a teacher approves, corrects or rejects it; until then the student sees only "complete, awaiting teacher".
**Status.** Approval flow deferred to Iteration 2; reviews currently gate content only. The student view already never carries a diagnosis, posterior, rule or correct answer (`sonder_api/student_view.py`, tested).

## R5 — Announcements and messaging (functional)

**Acceptance.** A teacher can send a message or announcement and a permitted recipient can read it; role and class rules hold.
**Status.** Deferred; fixtures only, not in the Iteration 1 schema.

## R6 — Versioned, reviewed, frozen packs (functional; data owner Harris, rules Ahmed)

**Statement.** An item is approved only by a recorded review from someone other than its author; a frozen, hashed pack is immutable; the engine reads only frozen packs.
**Acceptance.** Invalid edits are rejected (75 negative schema tests, as reported by the schema author); Maths_v2 loads (117 items, 468 options, 114 items in seven units); rule V5′ holds in every unit; all 28 misconceptions are used.
**Status.** Partial. The schema rules, pack loader and a CI import check exist (75 rejected writes, 15 of 15 checks, all 117 items load); the production import into the hosted database is not done; the three unplaced items, the link-grade column and a bank-wide human spot-check are open.

## R7 — Latency (non-functional)

**Acceptance.** One posterior update plus selection takes under 300 ms.
**Measured.** Median 9.9–11.7 ms and 95th percentile 22.8–23.6 ms per full step on units U1, U4 and U7 (Table 7.4); an earlier run gave medians of 7.0–8.5 ms.
**Status.** Met for the library; the database and network are not included.

## R8 — Determinism and safe input handling (non-functional)

**Acceptance.** Same pack, parameters and answers give identical posteriors, selections, reasons and decisions; an invalid option raises `invalid_option`; zero evidence raises `zero_evidence`; a rejected request leaves the state unchanged; parameters are recorded in every trace.
**Status.** Met for the engine and the service; pinning parameters per session in the database is not yet integrated.

## R9 — Live student diagnostic (functional; owner Ahmed)

**Statement.** A student takes a Mathematics diagnostic in the interface that is computed by the real engine, can resume after a refresh, and never sees the result.
**Acceptance.** Answers in the browser produce the engine's next item; a refresh resumes the same question; replies carry no diagnosis, posterior, rule or correct answer; a lost reply is retried without counting the answer twice; the D1 and D2 sessions match the engine.

**Evidence.** `sonder_api/`; 16 service tests; 123 interface tests; `scripts/check_engine_ui.py`.
**Status.** Met as a local checkpoint. **Gaps.** Runs are held in memory; no login; no teacher result view.

# Proposal commitments not yet in R1–R9

Accuracy and calibration targets (at most 5% confident misdiagnoses with at least 70% coverage; $\kappa \geq 0.7$) are untestable without real data. The evidence trail (every question, ranking, answer, posterior and reason, append-only) is supported by the schema and by the trace module; storing it is not yet integrated. Privacy and security (pseudonymous identifiers, role-based access) apply to the product layer. Low-bandwidth usability (about 2 Mbps) and a usability score are deferred.

# Appendix C

# Detailed Test Cases

## C.1 Deterministic students through the run loop

Each student always chooses the first option linked to its misconception (or the correct option for NONE) and never slips. Outcome: D diagnosed, N none_detected, S escalated_no_separator, X escalated_exhausted; the number is the count of answers used. These are the results of running the engine on the evaluation branch; `tests/test_engine_students.py` asserts the totals.

| Unit | Student (MATH id): outcome and answers |
|---|---|
| U1 | 1248: D2; 1379: D3; 1988: D3; 2392: D2; NONE: N3 |
| U2 | 61: D3; 658: D2; 1715: D2; 1965: D3; 2039: D2; NONE: N3 |
| U3 | 2138: D3; 2206: D3; 1074: S3; 1651: S3; 2359: D3; NONE: N3 |
| U4 | 126: D3; 228: D2; 1416: D3; 1514: S2; 1773: S2; 2093: X5; NONE: N3 |
| U5 | 414: D2; 734: D2; 952: D3; 1072: D2; NONE: N3 |
| U6 | 1074: S3; 161: D2; 767: D2; 1651: S3; 2359: X4; NONE: N3 |
| U7 | 2359: D2; 1639: D4; 2093: D2; 2271: D3; 1514: S3; 1715: D2; 1773: S3; NONE: N4 |

Totals: 26 D, 7 N, 8 S, 2 X (43 students); no student needs more than five answers.

## C.2 Injected faults (mutation check)

Wrong logarithm base; no rounding of EIG; reversed tie rule; worst item chosen; UNMODELLED allowed as a leader in the reason; >= instead of > in the separation test; wrong guess parameter. Each was caught by at least one test.

## C.3 Selection test classes (`tests/test_selection.py`)

| Class | Tests | Checks |
|---|---|---|
| EntropyTests | 4 | Uniform, deterministic and mixed distributions; zero terms |
| ExpectedInformationGainTests | 7 | Hand values; agreement with the update route; bounds; clamp and error for negative values |
| SelectNextTests | 9 | Highest EIG wins; ties by pack order; ranking order; empty list; rounding |
| SelectionReasonTests | 9 | Template fields; pair rule with total variation strictly above $\delta$; tie sentence; fallback sentence |
| SpecWorkedExampleTests | 2 | D1 and D2 choices, EIGs and posteriors to four decimals |
| IndependentRealBankChecks | 6 | 405 exact-fraction EIG values; 300 random posteriors against brute force; bounds; shuffled order; scripted sessions for U1–U5 |
| Eligibility checks | 3 | Twin partners and asked items excluded; pack order preserved |

## C.4 Commands to reproduce

```
python -m unittest discover -s tests -v
python -m unittest discover -s tests -p "test_selection.py" -v
python -m sonder_pack.run_session --unit U1 --answers D,B,B,A --table
```

# Appendix D

# Repository and Contribution Evidence

- Repository: https://github.com/sonder-fyp/SONDER (created 9 October 2026). Evaluation commit: 2c266d47472e130eea2fb68ae1c19972d0994cee on dev, 12 October 2026 (code freeze for the live demonstration); no release tag exists yet.
- Pull requests #1 to #38 (35 merged into dev); see Tables 8.2 and 8.1. The list is at github.com/sonder-fyp/SONDER/pulls.
- CI: workflow .github/workflows/ci.yml with jobs ci (interface), math-data (validator and its tests), engine (engine tests), db (schema checks in a throwaway PostgreSQL container) and an engine API workflow. Latest run on dev: https://github.com/sonder-fyp/ (success). Schema checks run: https://github.com/sonder-fyp/SONDER/actions/runs/3817526
- Task identifiers T00 to T14 are defined in ITERATION_1_TEAM_WORK_DIVISION_MATH_ONLY.md; no task board or GitHub issues are used, and the pull requests that use the template name their task IDs.
- Review: each pull request is reviewed by the reviewer the work division names (Harris reviews Ahmed's, Ahmed reviews Ashher's, Ashher reviews Harris's); review comments are in the pull-request threads.

# Appendix E

# Additional Design Diagrams

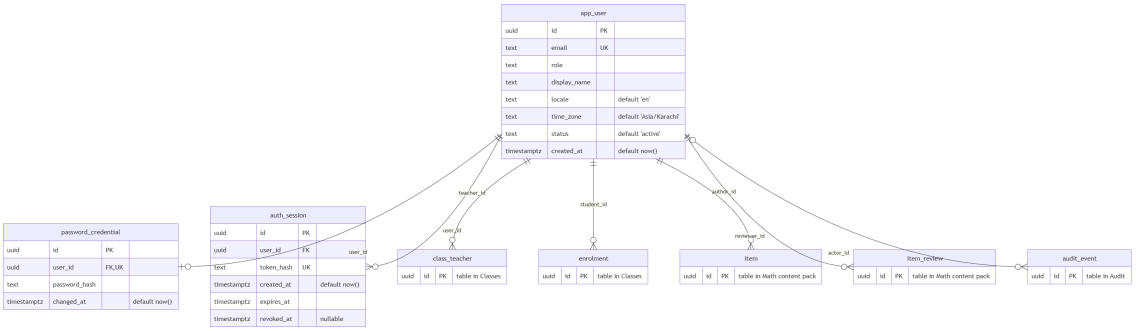



Figure E.1: ERD: users and access.

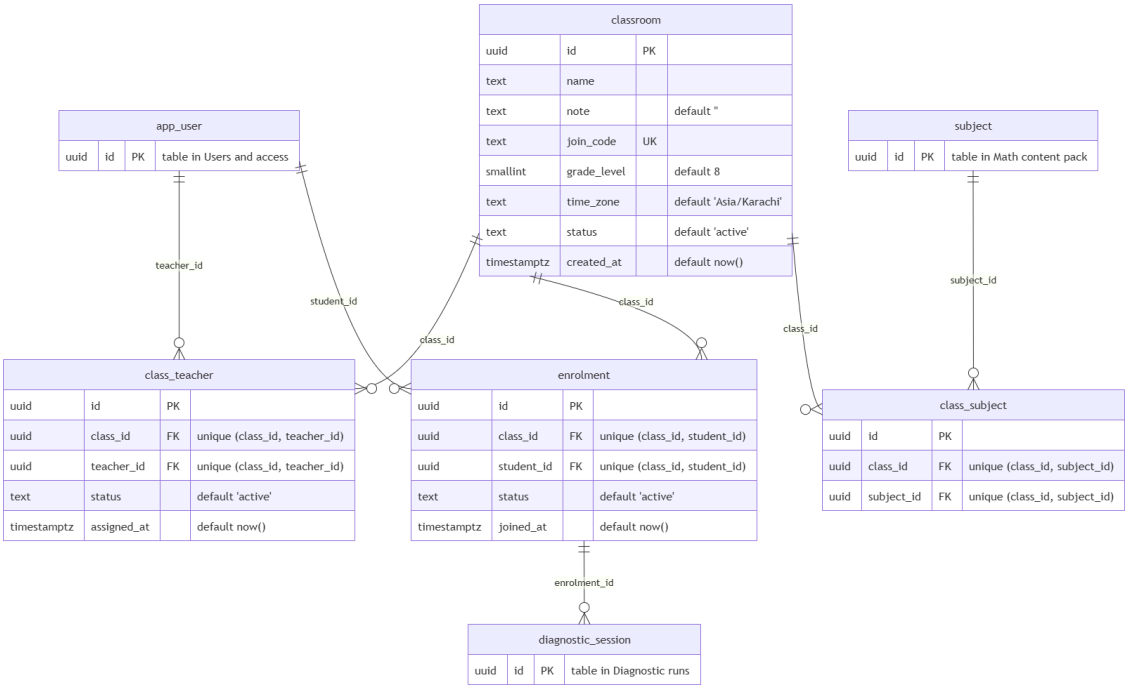



Figure E.2: ERD: classes and enrolment.

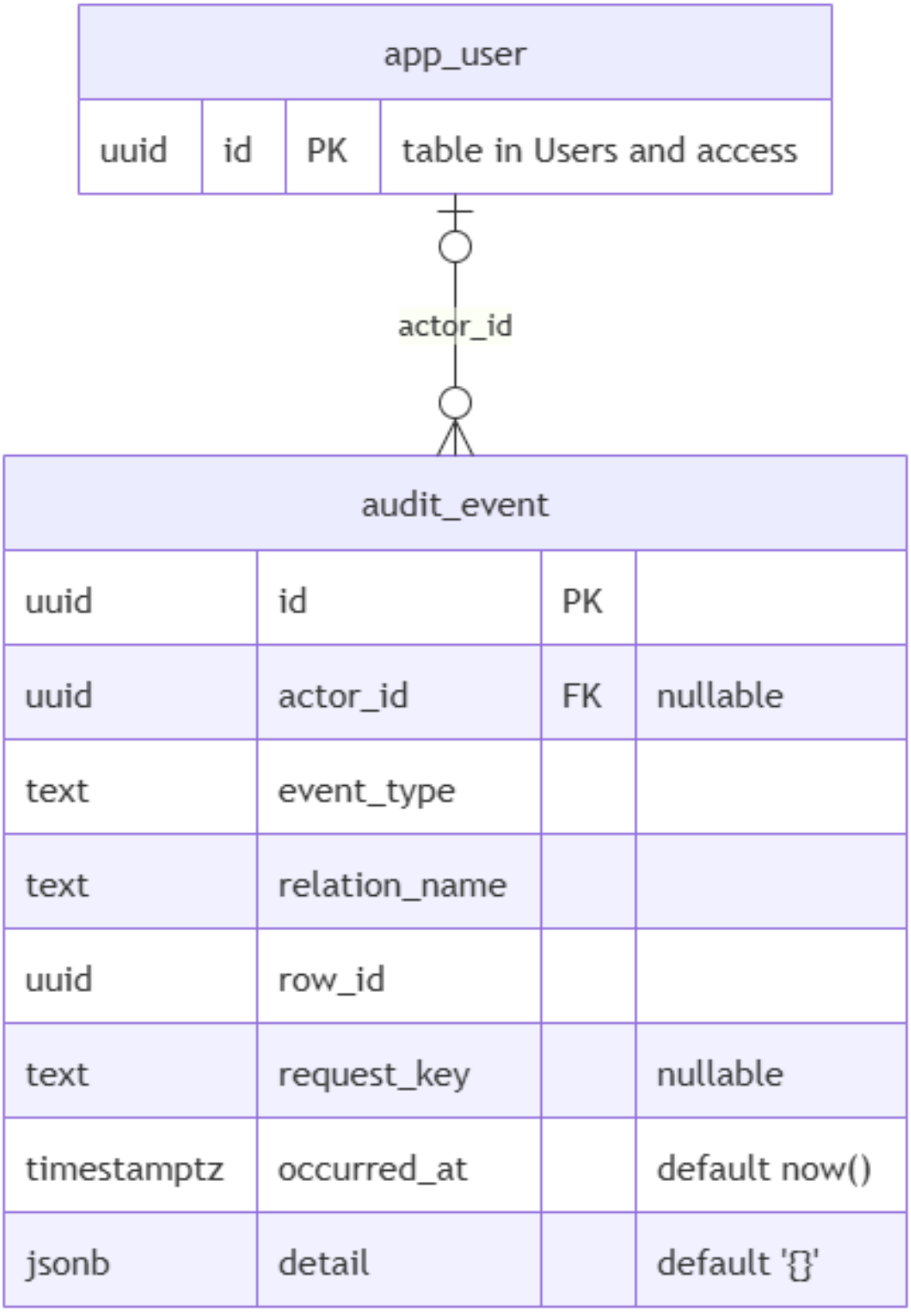



Figure E.3: ERD: audit.

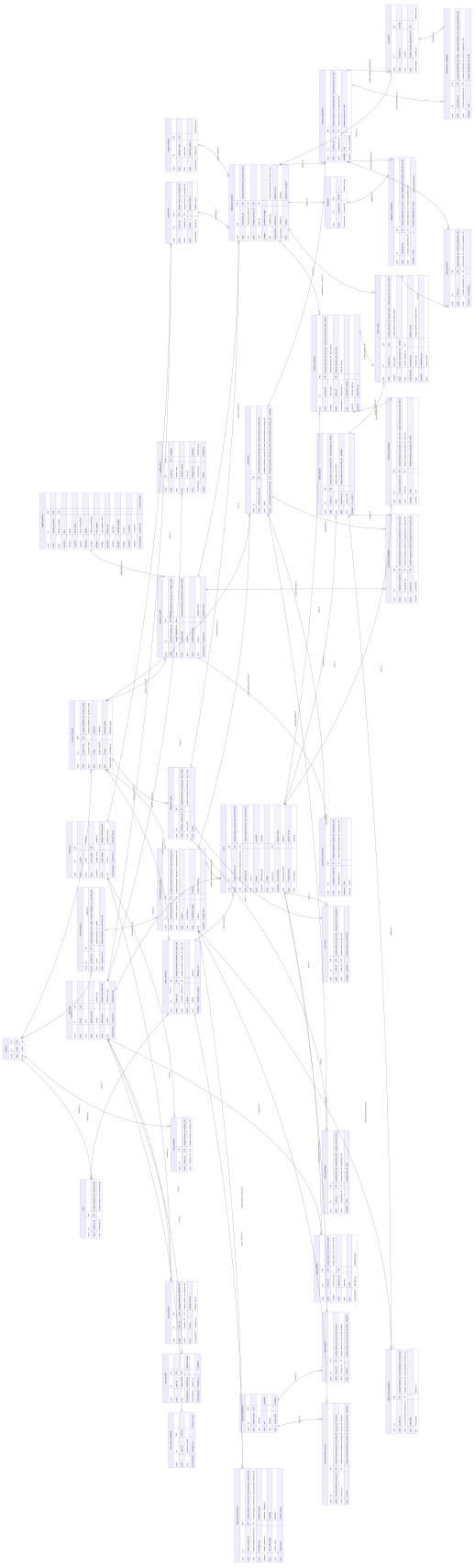


Figure E.4: Complete Iteration 1 schema (41 tables, revision 3).

# Appendix F

# Meeting and Progress Evidence

| Date | Meeting | Outcome |
|---|---|---|
| 9 September 2026 | Scope meeting with Dr. Ali Zeeshan Ijaz | Discussion held to improve the technical challenge; the agreed scope is to be written into the roadmap (docs/ROADMAP.md) |
| Throughout | Supervisor and team coordination | No meeting log is kept in the repository. Decisions are recorded in pull-request threads and in the specification decision list (D-1 to D-6, PR #28); ownership and reviewers are recorded in the work-division document |

# Bibliography


[1] Y. Cheng, "When cognitive diagnosis meets computerized adaptive testing: CD-CAT," *Psychometrika*, vol. 74, no. 4, pp. 619–632, 2009.

[2] K. K. Tatsuoka, "Rule space: An approach for dealing with misconceptions based on item response theory," *Journal of Educational Measurement*, vol. 20, no. 4, pp. 345–354, 1983.

[3] Eedi, "Mining misconceptions in mathematics," Kaggle competition, 2024, misconception catalogue used to seed the Mathematics content; licence terms still to be recorded.

[4] Z. Wang, A. Lamb, E. Saveliev *et al.*, "Results and insights from diagnostic questions: The NeurIPS 2020 education challenge," 2021.

[5] I. A. Halloun and D. Hestenes, "The initial knowledge state of college physics students," *American Journal of Physics*, vol. 53, no. 11, pp. 1043–1055, 1985.

[6] R. Driver, A. Squires, P. Rushworth, and V. Wood-Robinson, *Making Sense of Secondary Science: Research into Children's Ideas*. Routledge, 1994.

[7] D. F. Treagust, "Development and use of diagnostic tests to evaluate students' misconceptions in science," *International Journal of Science Education*, vol. 10, no. 2, pp. 159–169, 1988.

[8] B. Andersson, "Pupils' conceptions of matter and its transformations (age 12–16)," *Studies in Science Education*, vol. 18, pp. 53–85, 1990.

[9] C. E. Shannon, "A mathematical theory of communication," *Bell System Technical Journal*, vol. 27, no. 3, pp. 379–423, 1948.